



UNIVERSITY OF TARTU



Enterprise System Integration (MTAT.03.229)

LECTURE 9: SECURITY ASPECTS OF INTEGRATION - I

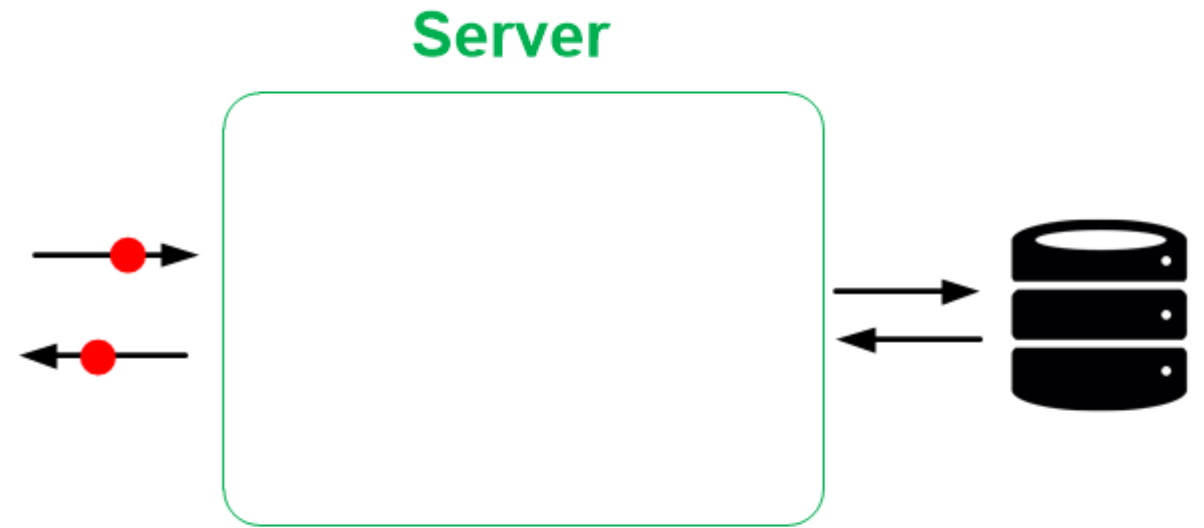
**MOHAMAD GHARIB
UNIVERSITY OF TARTU**

Info. System Security: a Short History

- 1960s - Physical security:** security was limited to the access points, where computers were stored.
- 1970s - Evolution of personal computer and information flow through telephone lines¹:** first hackers appeared, who tries different ways to intercept such information to steal the data.
- 1980s - Evolution of cyber-crime:** hacking and other forms of cyber crimes skyrocketed as hackers find various methods to break into the computer systems benefiting from the absent of strict regulation against hacking/cyber-crimes.
- 1990s - Hacking becoming a serious concern:** As the worldwide web becomes available, people started putting their personal information online; hackers saw this as a potential revenue source. Although firewalls/antivirus programs exists, yet the web was a mostly unsecured.
- 2000s - Cybercrime becoming a serious issue:** hacking was not considered as serious issues but with evolution of hacking methods and the severity of their consequences, governments started taking strong measures against cyber criminals.
- 2010s - now: Information security nowadays:** different protection measures/methods are actively developed but “hackers” are also developing new more sophisticated attack methods to breach such protection measures/methods.

¹ Only large organizations and governments were starting to link computers via telephone lines

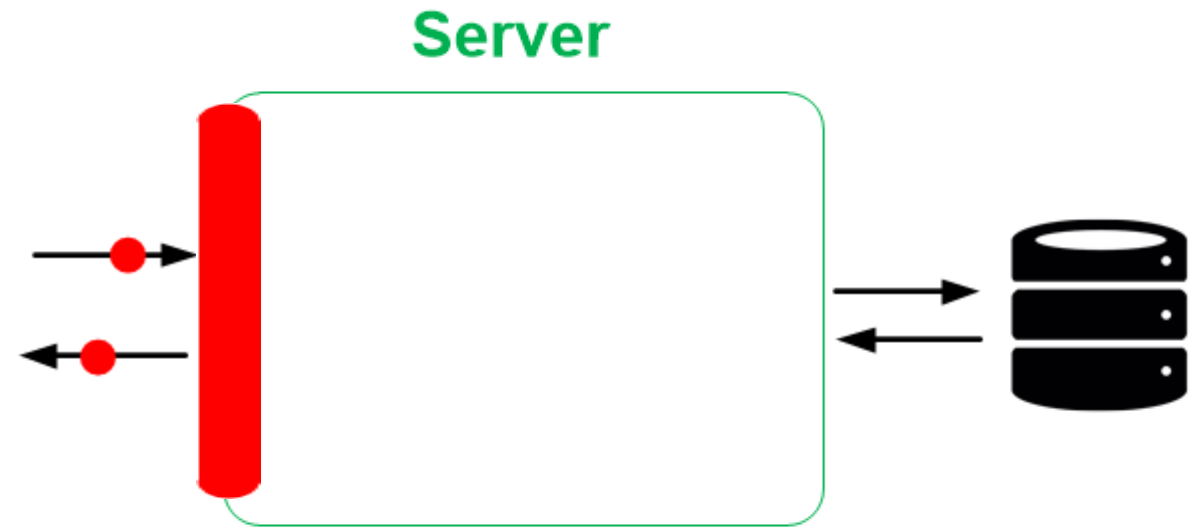
What is Security for our applications?



What is Security for our applications?

Who is asking for the recourse/service?

Is it allowed to “use” the recourse/service?



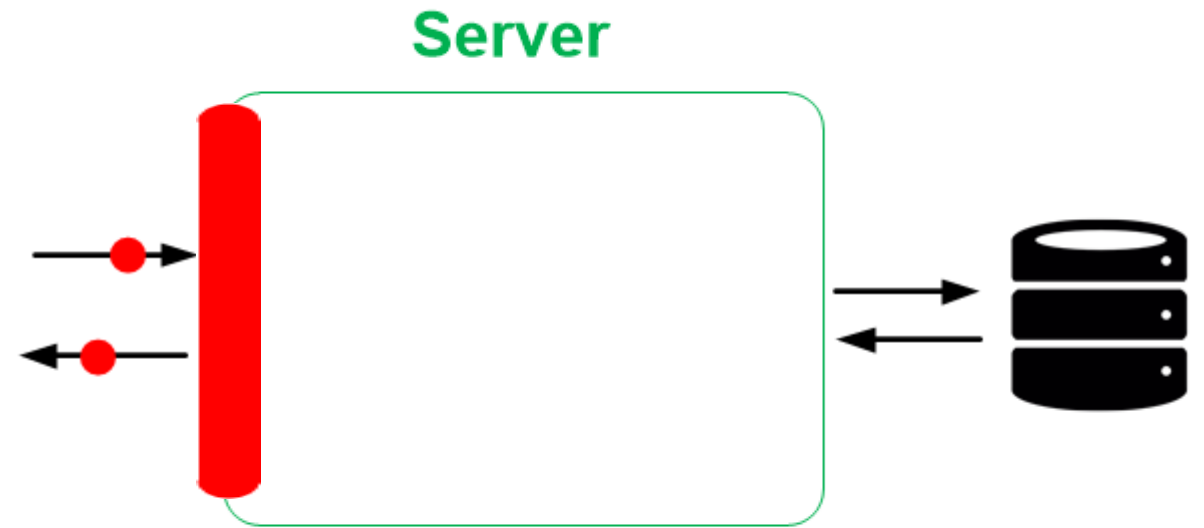
What is Security for our applications?

Who is asking for the
recourse/service?

Authentication

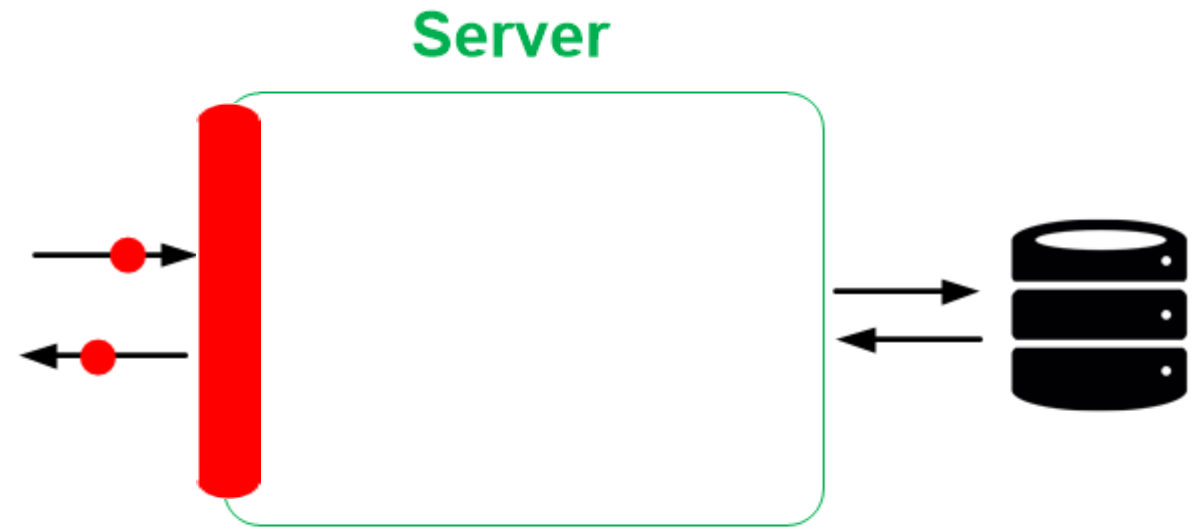
Is it allowed to “use” the
recourse/service?

Authorization



Four Core Spring Security Concepts

1. Authentication,
2. Authorization,
3. Granted Authorities/Roles,
4. Principle.



1. Authentication

Authentication is the process of verifying that
"you are who you say you are"

Authentication factors:

Knowledge based authentication (*something you know*): e.g., a password, partial password, personal identification number (PIN), security question, etc.

Ownership/possession-based authentication (*something you have*): e.g., ID card, security token, implanted device, cell phone with a built-in hardware token, software token, etc.

Inherence-based authentication (*something you are*): e.g., fingerprint, retinal pattern, signature, face, voice, unique bio-electric signals, other biometric identifiers, etc.

1. Authentication

Authentication is the process of verifying that
"you are who you say you are"

Authentication can be:

Single-factor authentication is the weakest level of authentication, only a single component from one of the three categories of factors is used to authenticate an individual's identity.

Multi-factor authentication involves two or more authentication factors. Clearly, two-factor authentication is a special case of multi-factor authentication.

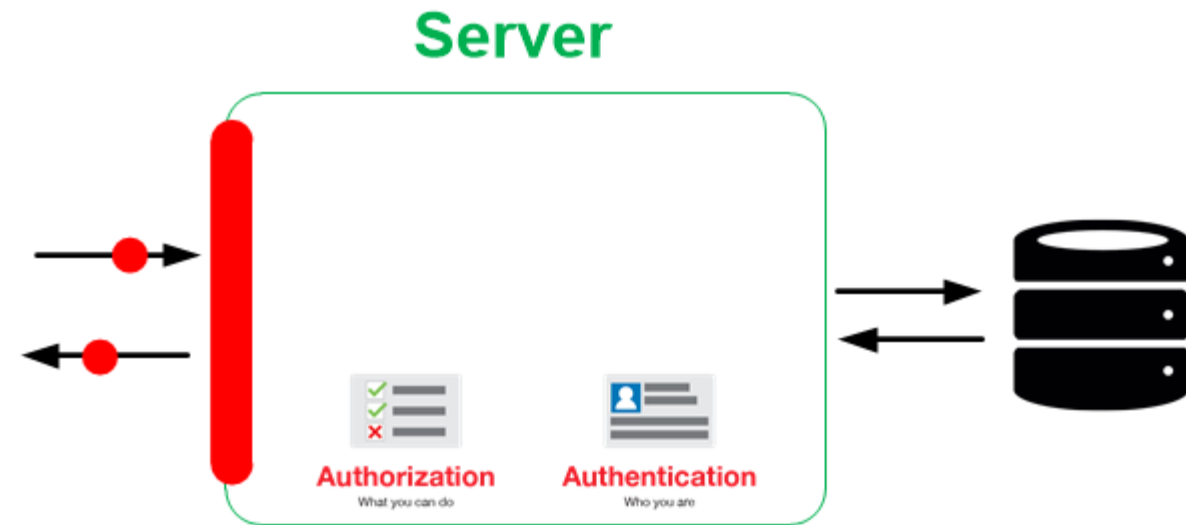
Note. Research suggests that for an effective authentication, at least two, and preferably all three, factors should be verified.

2. Authorization

Authorization is the process of verifying that
"you are permitted to do what you are trying to do".

In CS, **Authorization** is the function/process of specifying access rights/privileges to resources.

Authorization must always follow **authentication**: users should first prove that their identities are genuine before the system grant them access to the requested resources.



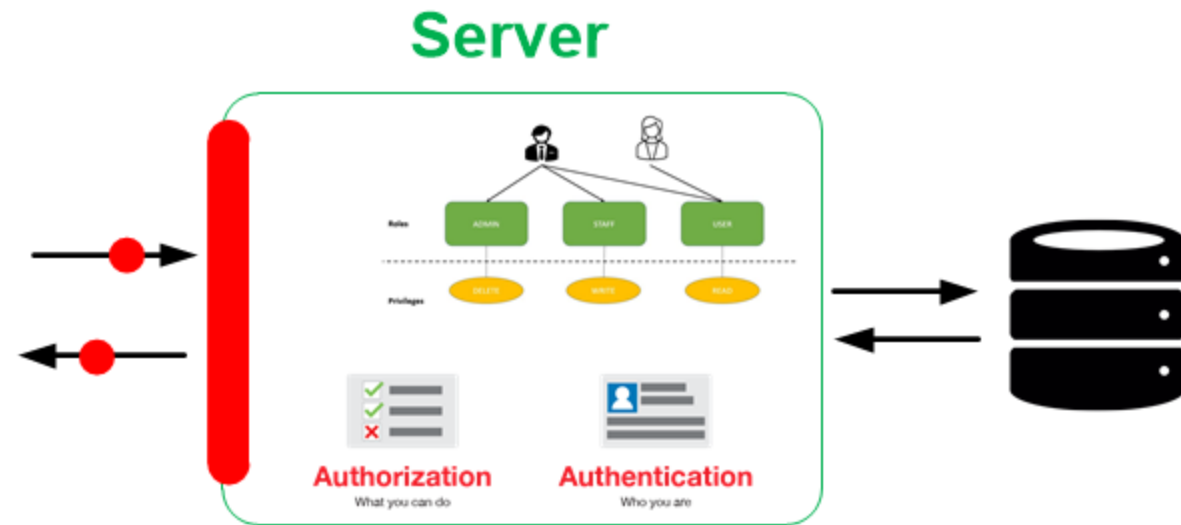
3. Granted Authorities/Roles

A **Granted Authority** can be seen as an individual privilege. Examples could include (e.g., read, write, execute, etc.).

Granted Authorities restrict/control access in a fine-grained manner.

A **Role** can be seen as containers of authorities/privileges

A **Role** restrict/control access in a coarse-grained *Granted Authorities*.

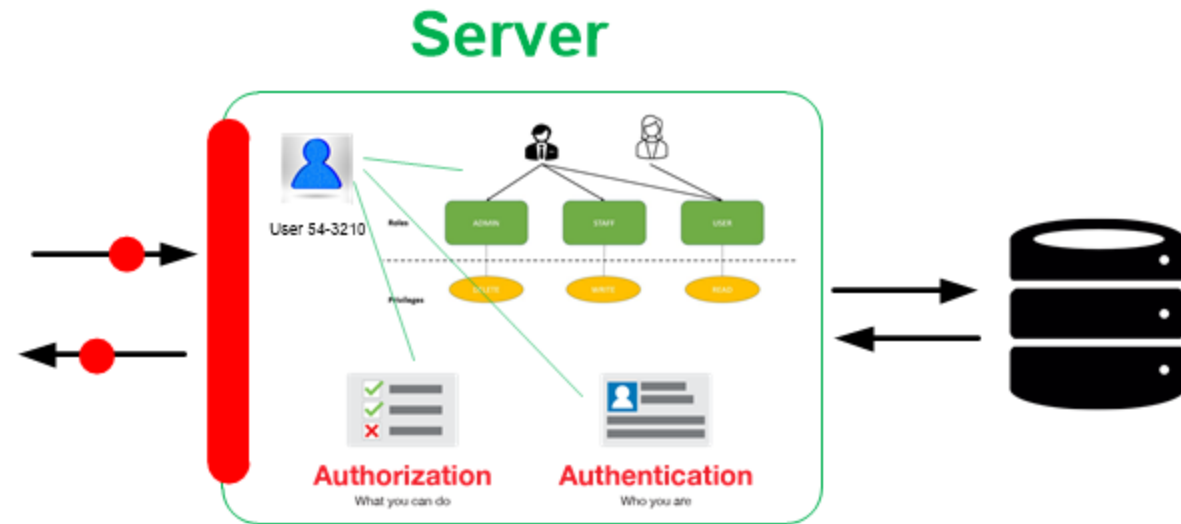


4. Principal

A **principal**, in computer security, is an entity that can be **authenticated**, and can be assigned rights and privileges/permissions over resources.

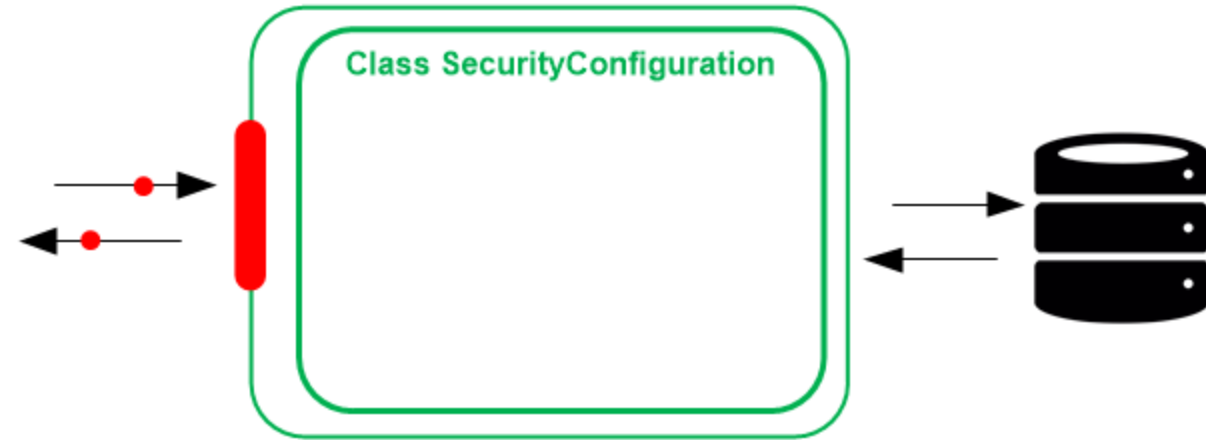
A **principal** can be an individual person, a computer, a service, etc.

A **principal** typically has an associated identifier that allows it to be referenced for identification or assignment of properties and permissions.



How can we secure our Spring Boot App?

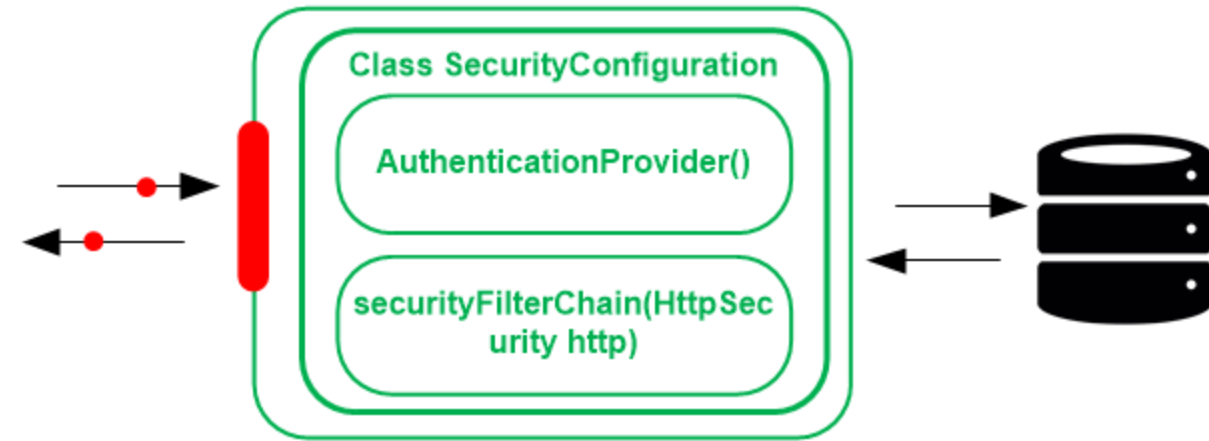
1. Configure a Spring Security authentication mechanism,
2. Configure a Spring Security authorization mechanism.



How can we secure our Spring Boot App?

1. Configure a Spring Security authentication mechanism,
2. Configure a Spring Security authorization mechanism.

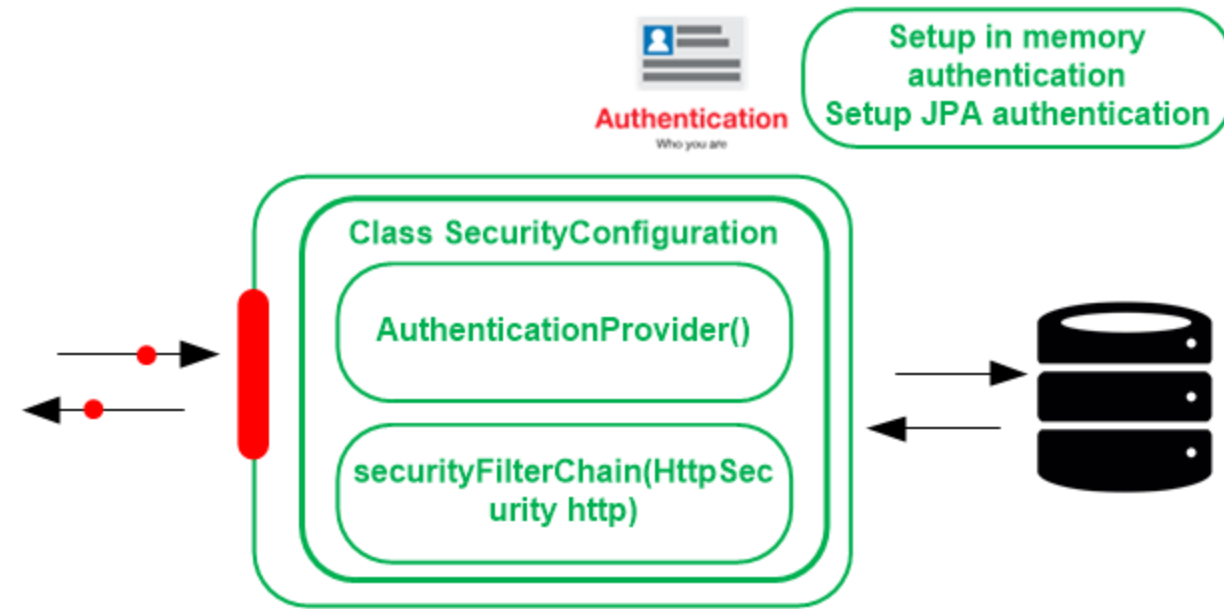
➡ Specify App users and associate appropriate roles to them, which define their allowed/prevented activities.



How can we secure our Spring Boot App?

1. Configure a Spring Security authentication mechanism,
2. Configure a Spring Security authorization mechanism.

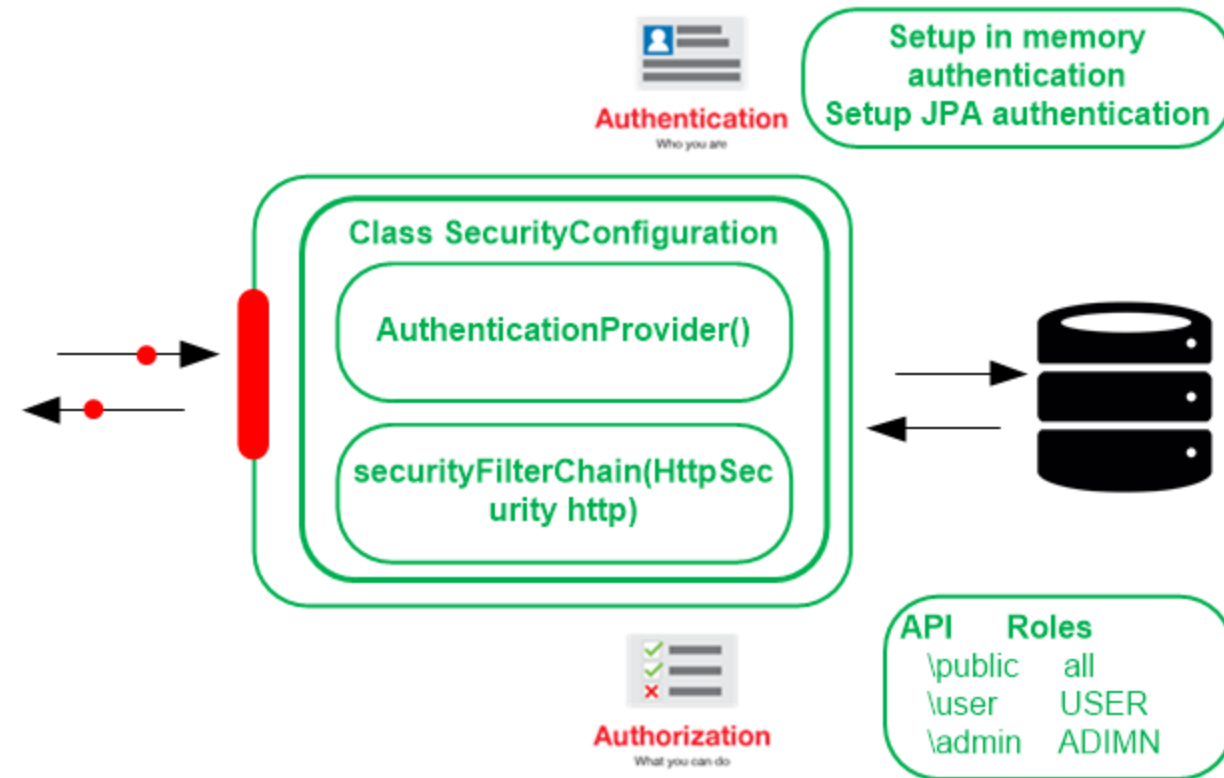
➡ Specify App users and associate appropriate roles to them, which define their allowed/prevented activities.



How can we secure our Spring Boot App?

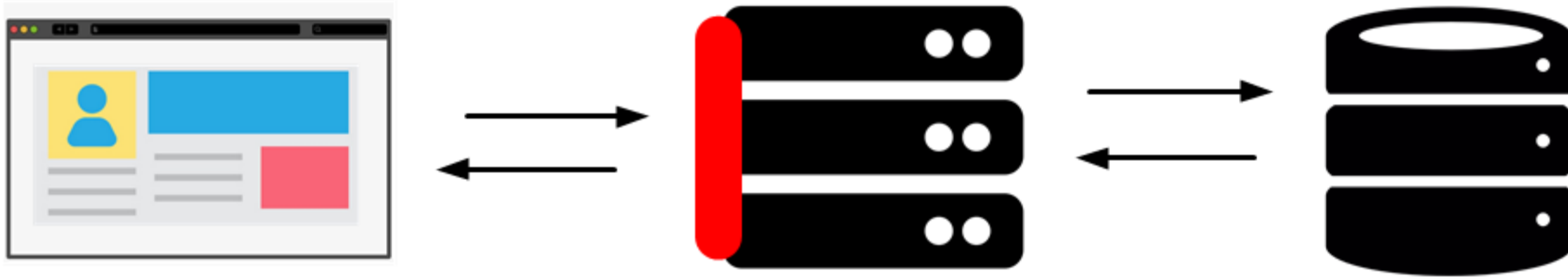
1. Configure a Spring Security authentication mechanism,
2. Configure a Spring Security authorization mechanism.

➔ Specify App users and associate appropriate roles to them, which define their allowed/prevented activities.



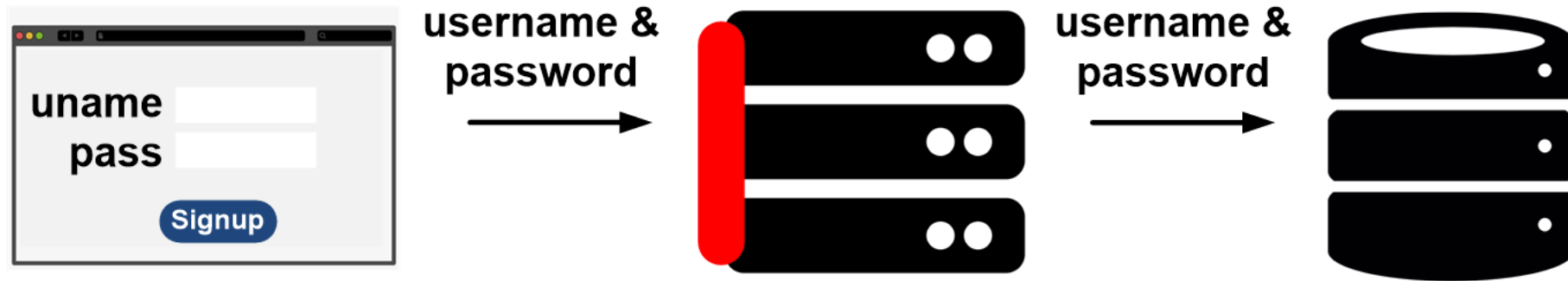
How authentication/authorization works?

How authentication works?



How I can authenticate a user?

How authentication works?

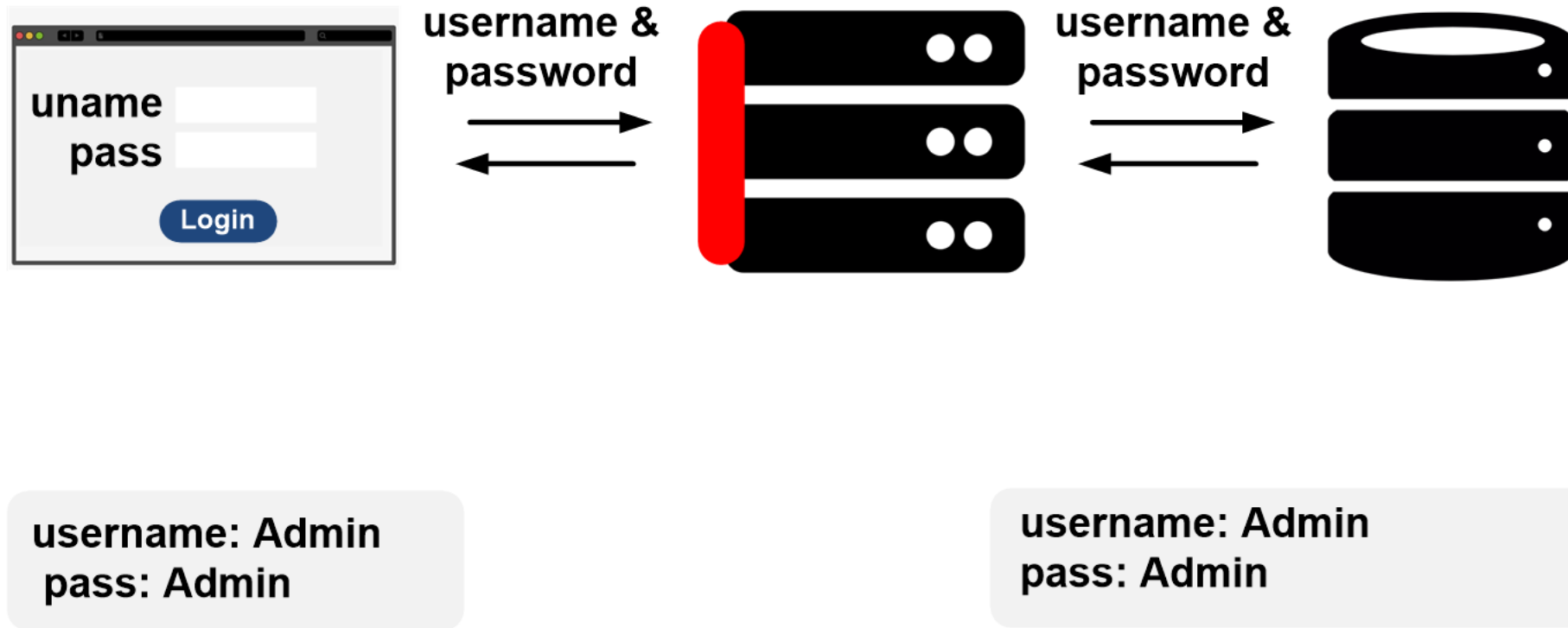


For in Memory auth/auth, we do not need a DB

username: Admin
pass: Admin

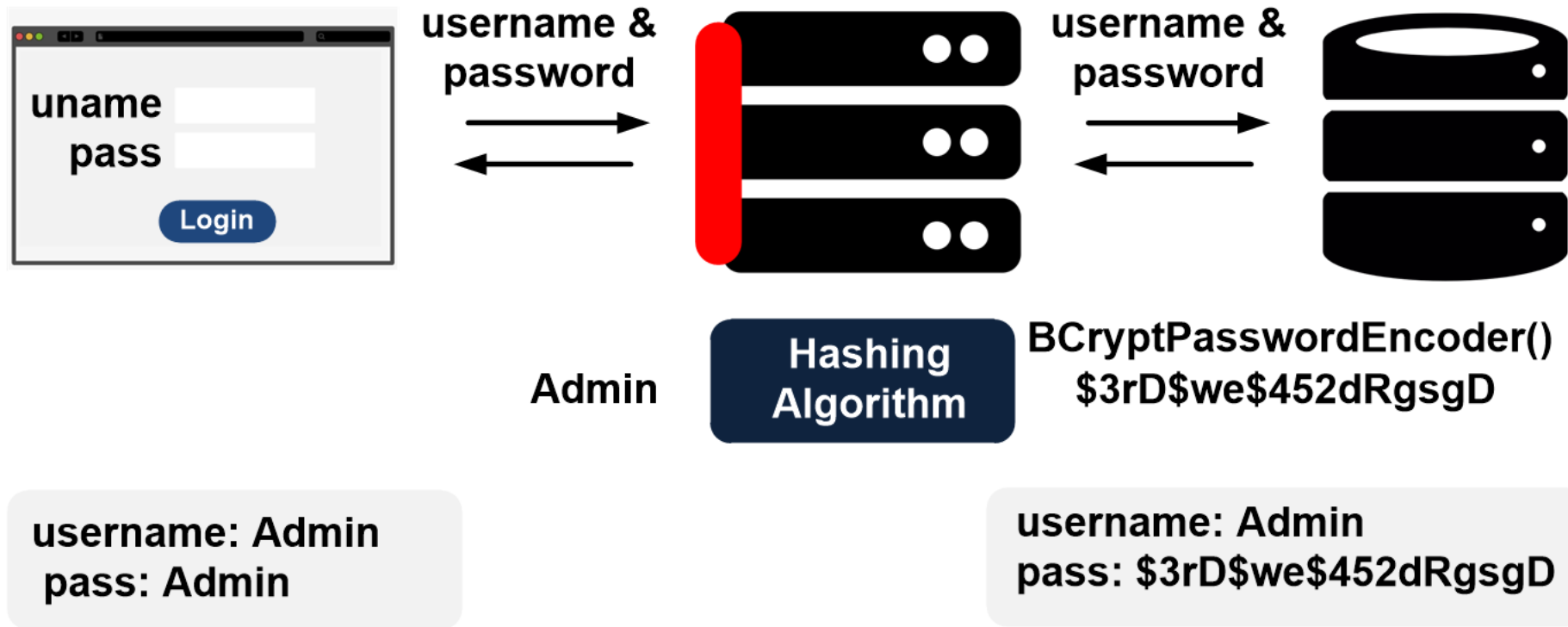
A user can **register/signup** by providing a **unique email** and a **password**, which can be used to **authenticate** the user.

How authentication works?



A user can **register/signup** by providing a **unique email** and a **password**,
which can be used to **authenticate** the user.

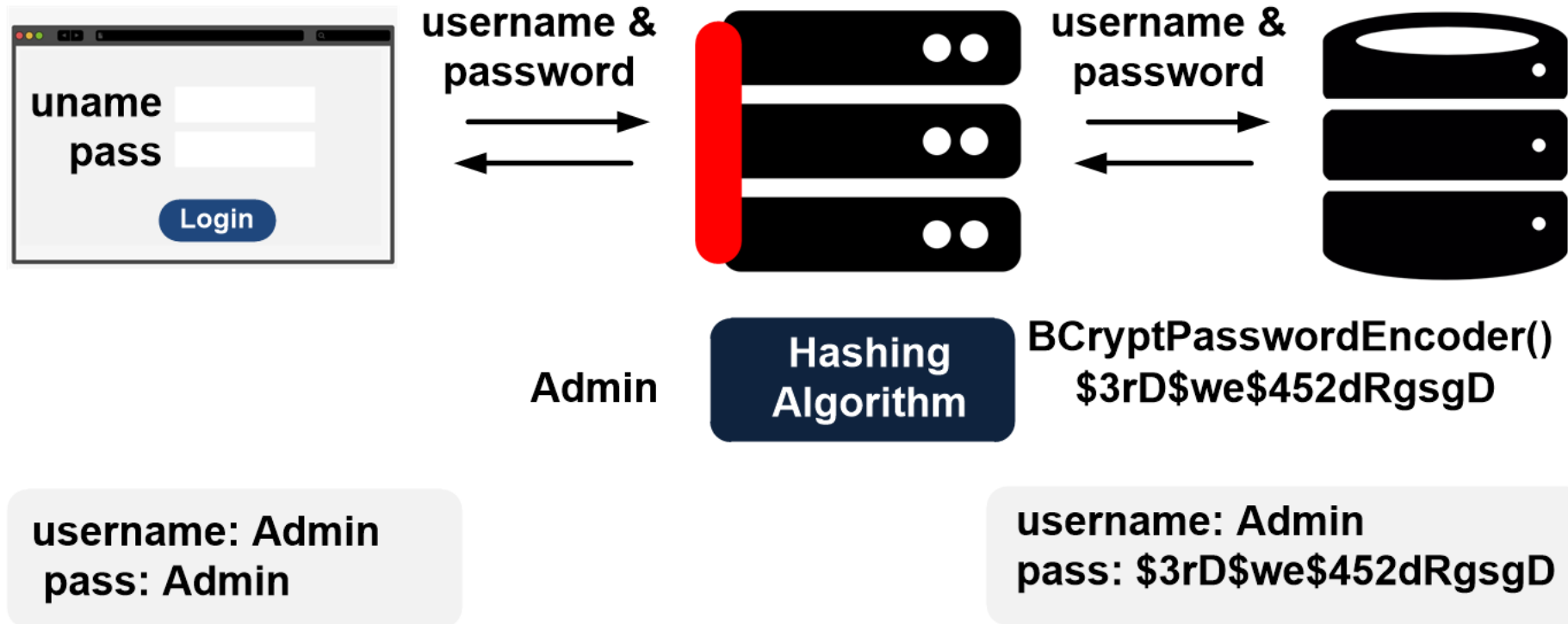
How authentication works?



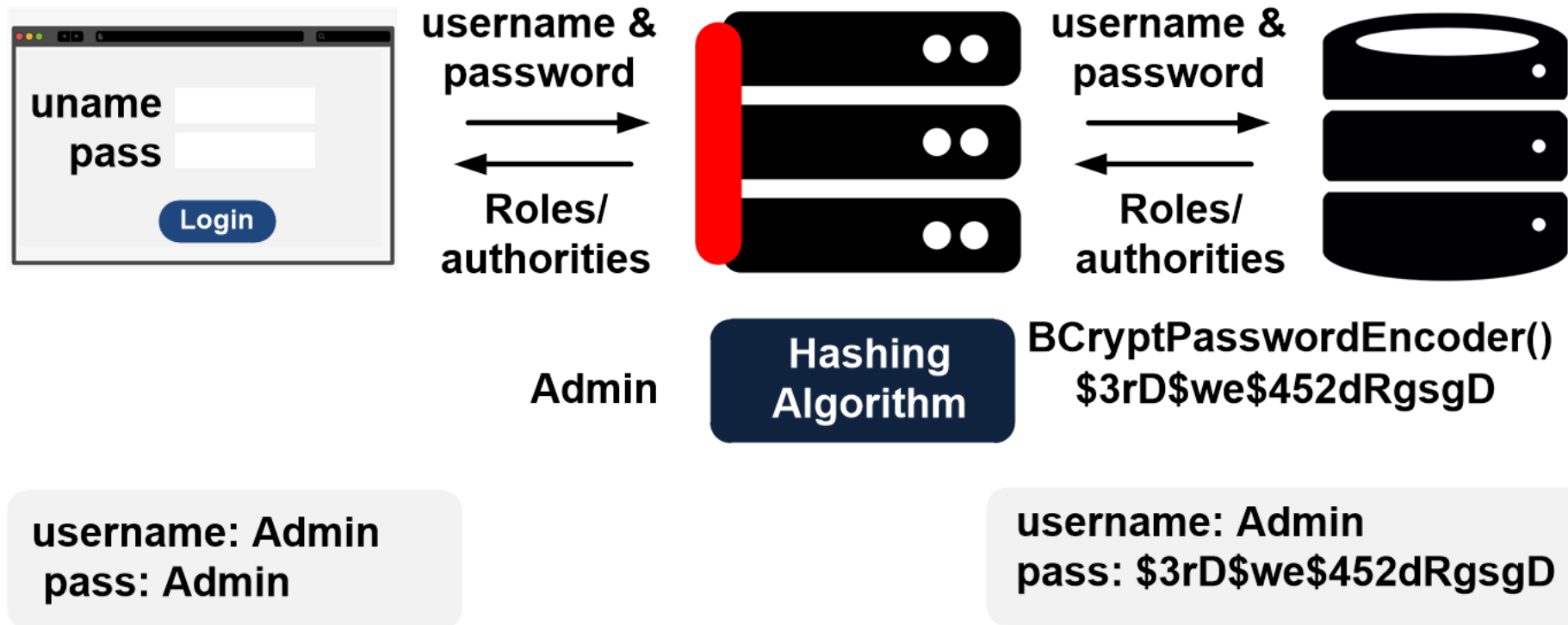
Hashing algorithms allows taking data of any size and apply a mathematical process to it to produce a unique output of string.

Regardless of the input length, a hash output is always of the same length.

How authorization works?



How authorization works?



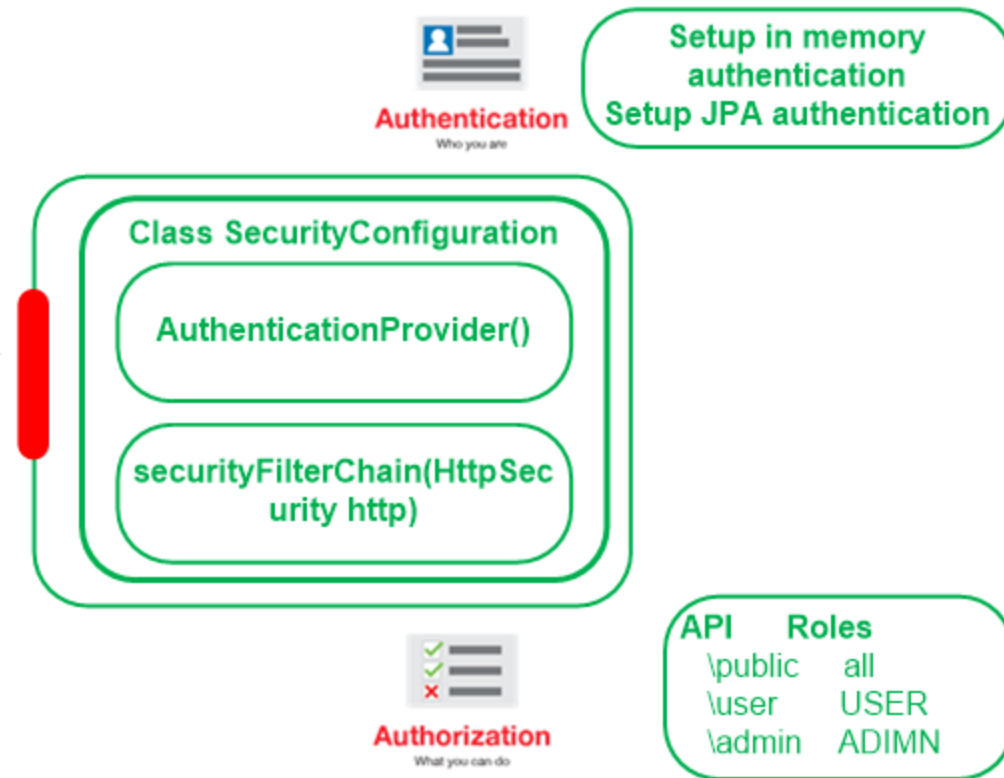
After a successful authentication, a user receives his **roles/authorities**, which can be used to **authorize** the user.

SB Security - in memory auth/auth

SB Security - in memory authentication

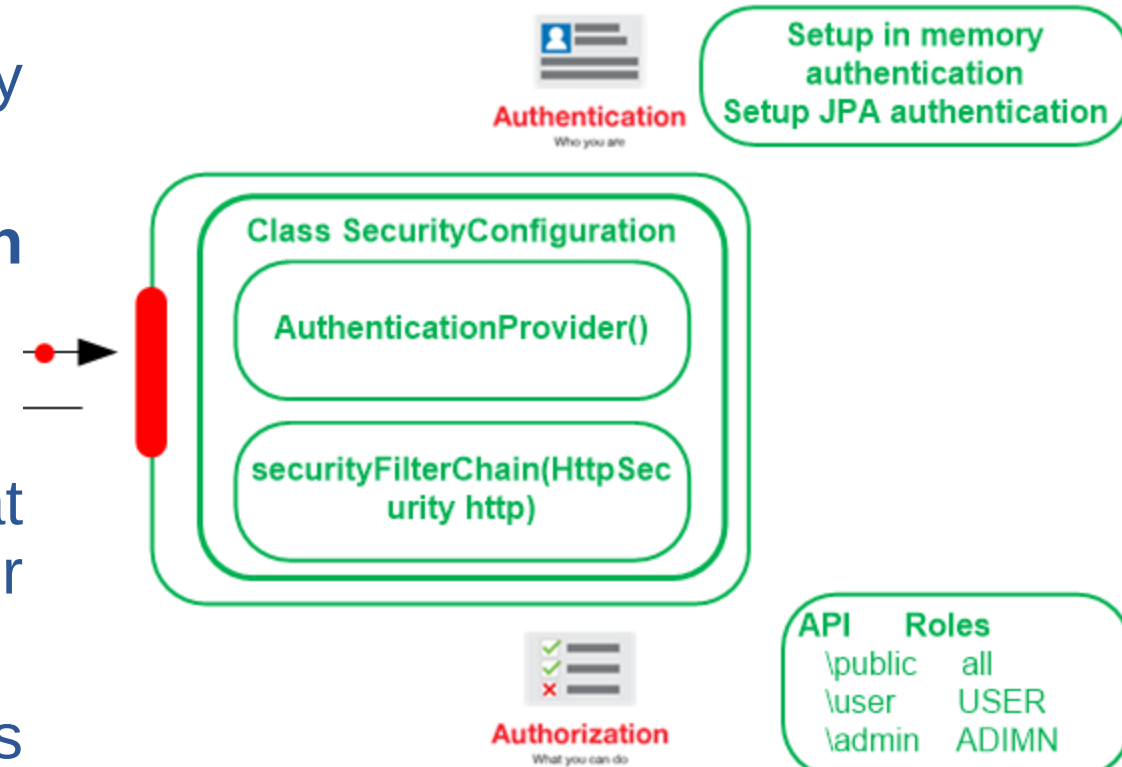
- **UserDetails** based authentication is used by Spring Security when it is configured to accept a username and password for authentication.
- **InMemoryUserDetailsManager** provides management of **UserDetails** by implementing **UserDetailsService** to provide support for username/password-based authentication that is stored **in memory**.
- To ensure that the password stored in memory is protected, we need to use **PasswordEncoder**.

➔ Specify App users and associate appropriate roles to them, which define their allowed/prevented activities.



SB Security - in memory authentication

1. Create a simple controller.
2. Configure a Spring Security in memory authentication mechanism.
3. You need to create **SecurityConfigeraion** class, in which you need to define:
 - A **PasswordEncoder**.
 - The **userDetailsService/userDetails** that contains the information of your users for auth.
 - **securityFilterChain(HttpSecurity http)** is used to configure the security filter chain.



SB Security - in memory authentication

```
// SecController
@RestController
@RequestMapping("/api")
public class SecController {

    @GetMapping("/public")
    public String publicAPI() {
        return "This is an unprotected endpoint";
    }

    @GetMapping("/admin")
    public String adminAPI() {
        return "Protected endpoint - only admins are allowed";
    }

    @GetMapping("/user")
    public String userAPI() {
        return "Protected endpoint - only users are allowed";
    }
}
```

SB Security - in memory authentication

```
// SecCo // SecurityConfig
@RestCor ...
@Request @Configuration
public class SecurityConfig {
    @EnableWebSecurity
    @EnableMethodSecurity

    @Get public class SecurityConfig {
        @Bean
        public PasswordEncoder passwordEncoder() {
            return new BCryptPasswordEncoder();
        }

        @Get public ...

        @Get public ...
    }
}
```

SB Security - in memory authentication

```
// SecCo // SecurityConfig
@RestCor ....
@Request @Configuration
public c @EnableWebSecurity
        @EnableMethodSecurity

    @Get public class SecurityConfig {
    publ     @Bean
            public PasswordEncoder passwordEncoder() {
            return new BCryptPasswordEncoder();    }

    @Bean
    @Get public UserDetailsService userDetailsService(PasswordEncoder encoder) {
    publ     UserDetails admin = User
            .withUsername("admin")
            .password(encoder.encode("admin"))
            .roles("ADMIN")
            .build();

    @Get
    publ     ...
            return new InMemoryUserDetailsManager(admin, user);
    }
    }
}
```

SB Security - in memory authentication

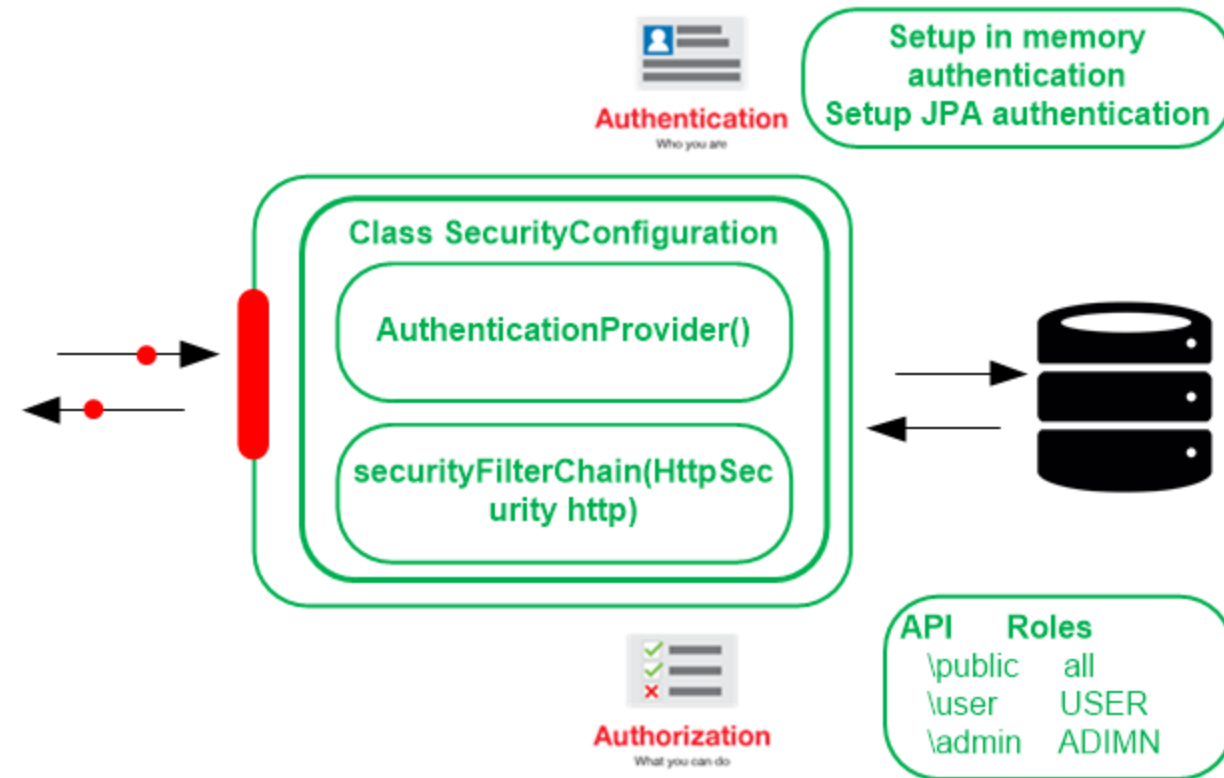
```
// SecCo // SecurityConfig
@RestCor ... // SecurityConfig
@Request @Con ...
public c @Ena @Bean
@Ena public SecurityFilterChain securityFilterChain(HttpSecurity http)
@Get publ throws Exception {
publ http
    .cors(withDefaults())
    // .csrf().disable()
    .authorizeHttpRequests(authorize -> authorize
    .requestMatchers("/api/public").permitAll()
    .requestMatchers("/api/admin").hasAnyRole("ADMIN")
    .requestMatchers("/api/user").hasAnyRole("USER")
    .anyRequest().authenticated()
    )
    .formLogin(withDefaults())
    .httpBasic(withDefaults());
    return http.build();
}
.. ..
```

SB Security auth/auth through JPA/DB

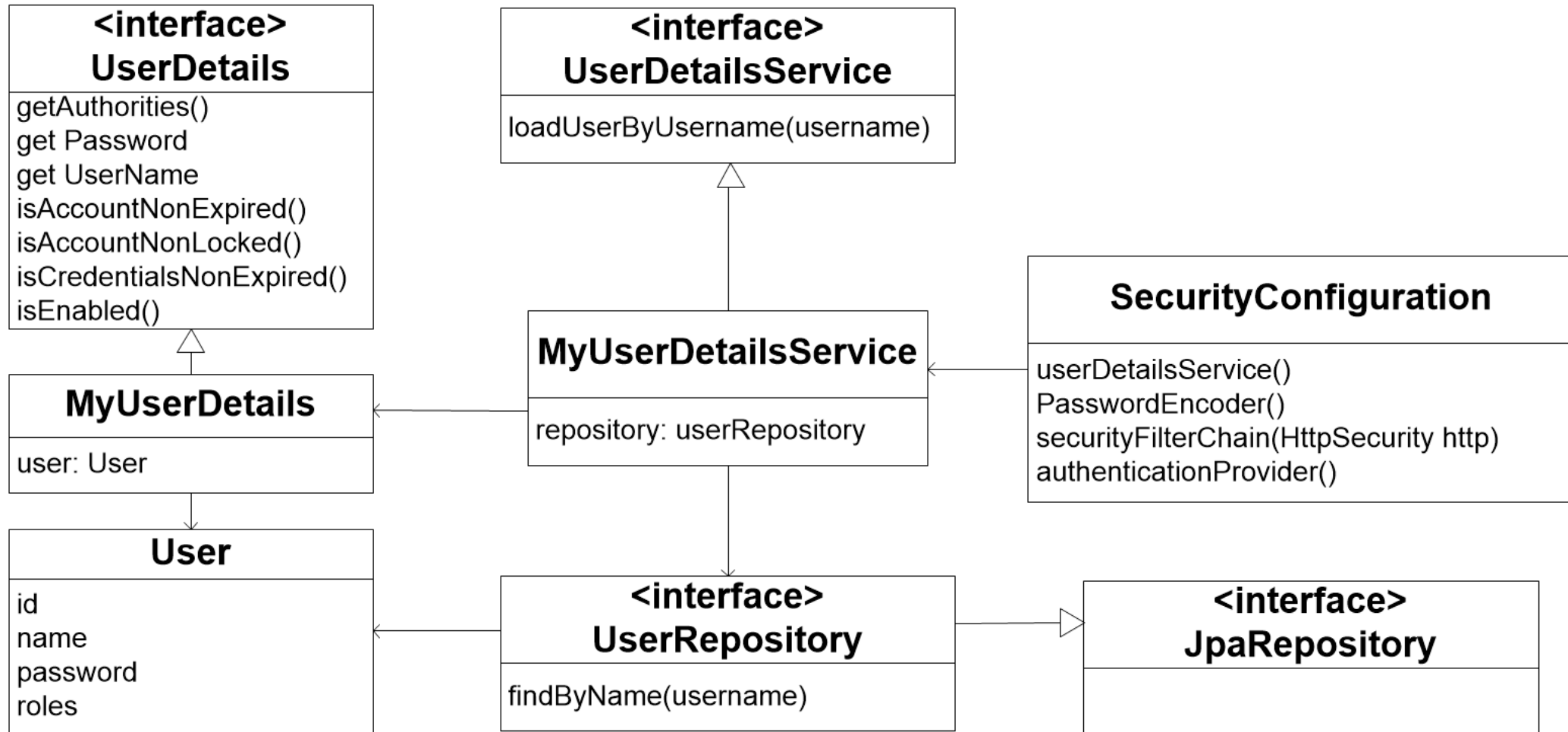
SB Security auth/auth through JPA/DB

- **UserDetails** is returned by the **UserDetailsService**.
- **UserDetailsService** is used by **DaoAuthenticationProvider** for retrieving a username, a password, and other attributes for authenticating with a username and password.
- **DaoAuthenticationProvider** also use a **PasswordEncoder**.

➔ Specify App users and associate appropriate roles to them, which define their allowed/prevented activities.

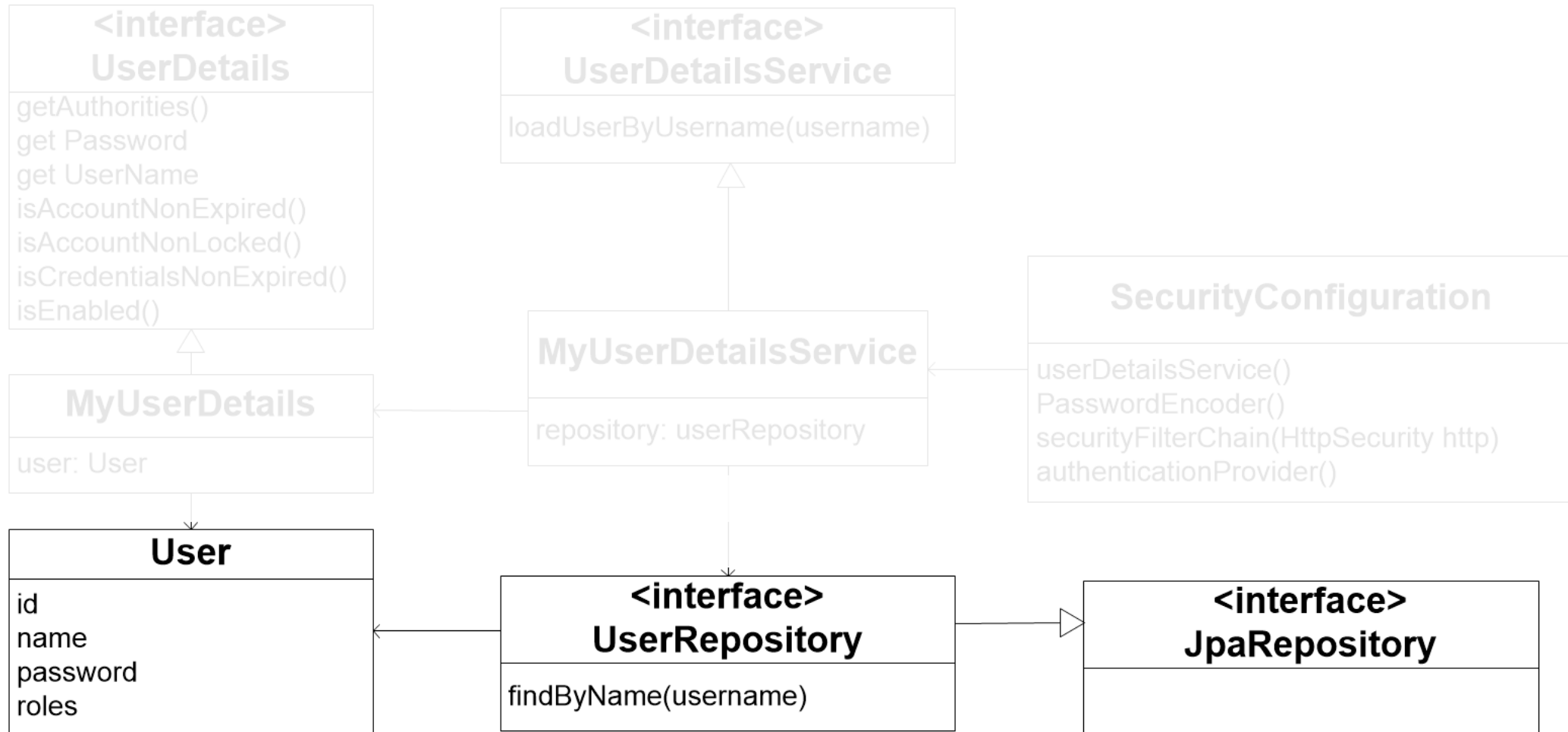


SB Security auth/auth through JPA/DB



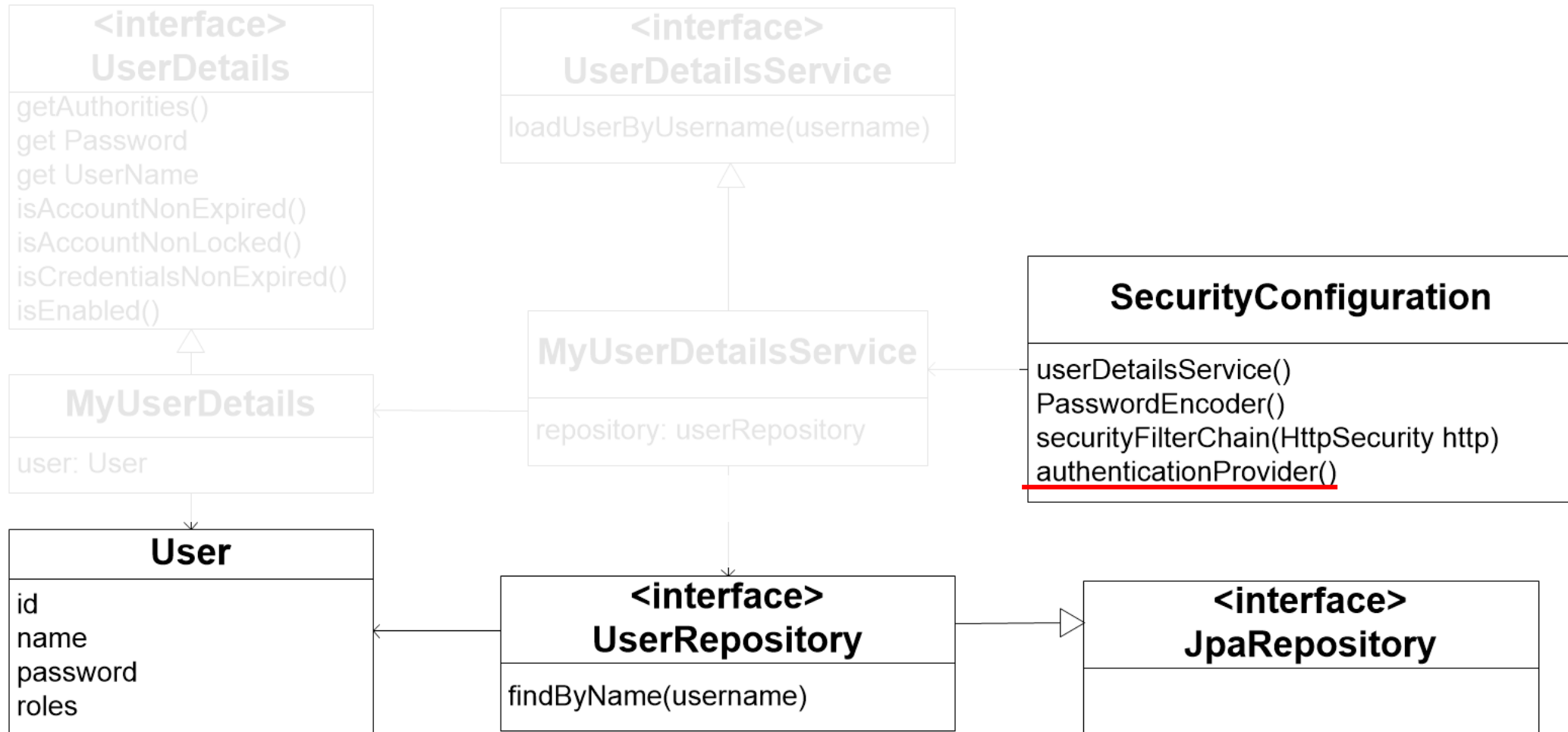
A metamodel of the classes and interfaces involved in the Authentication and Authorization through JPA /DB

SB Security auth/auth through JPA/DB



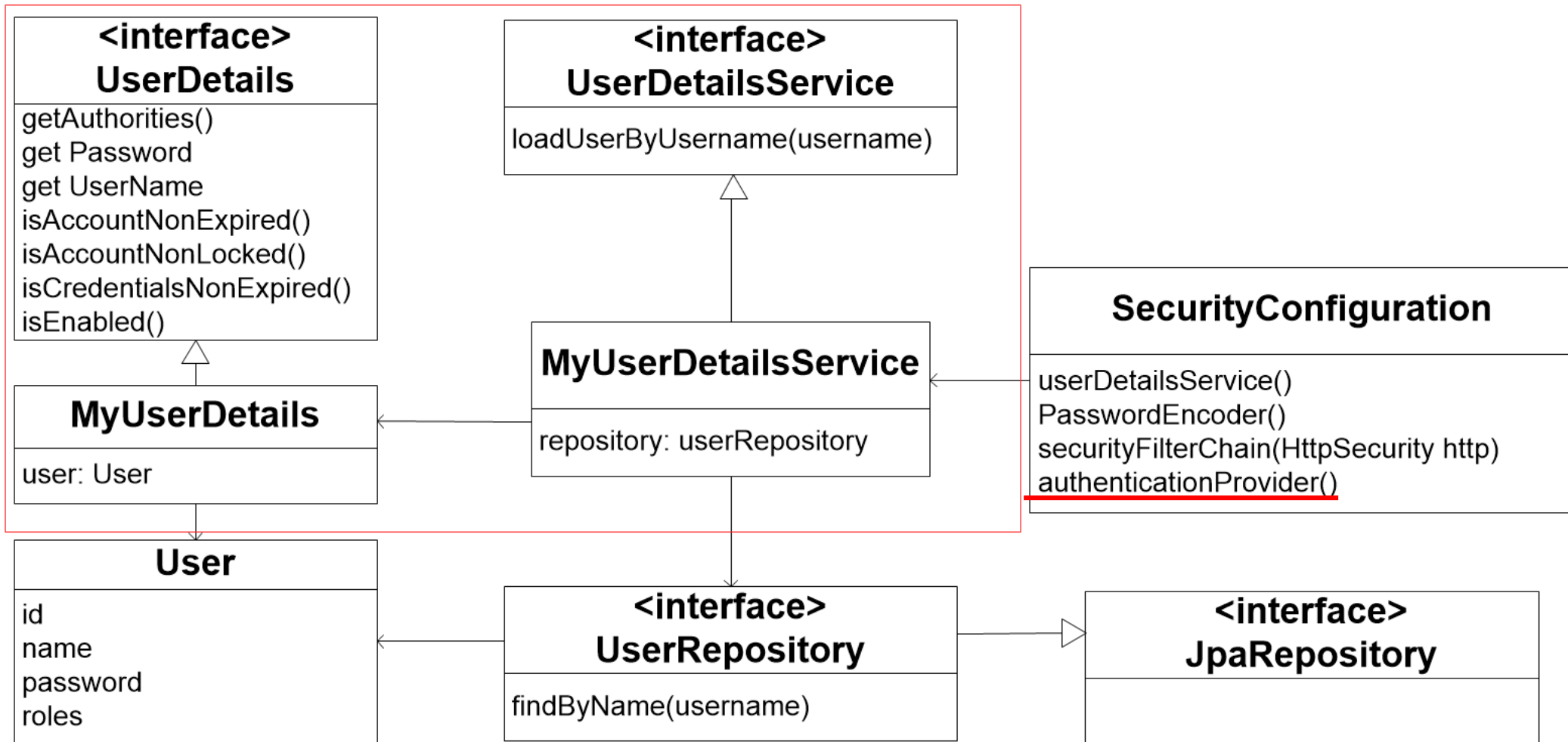
A metamodel of the classes and interfaces involved in the Authentication and Authorization through JPA /DB

SB Security auth/auth through JPA/DB



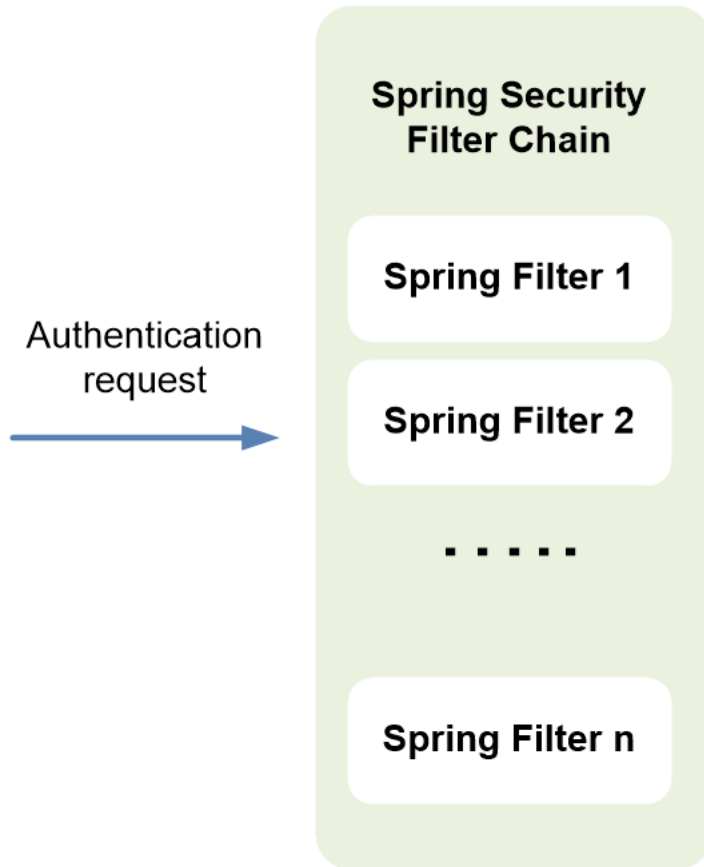
A metamodel of the classes and interfaces involved in the Authentication and Authorization through JPA /DB

SB Security auth/auth through JPA/DB

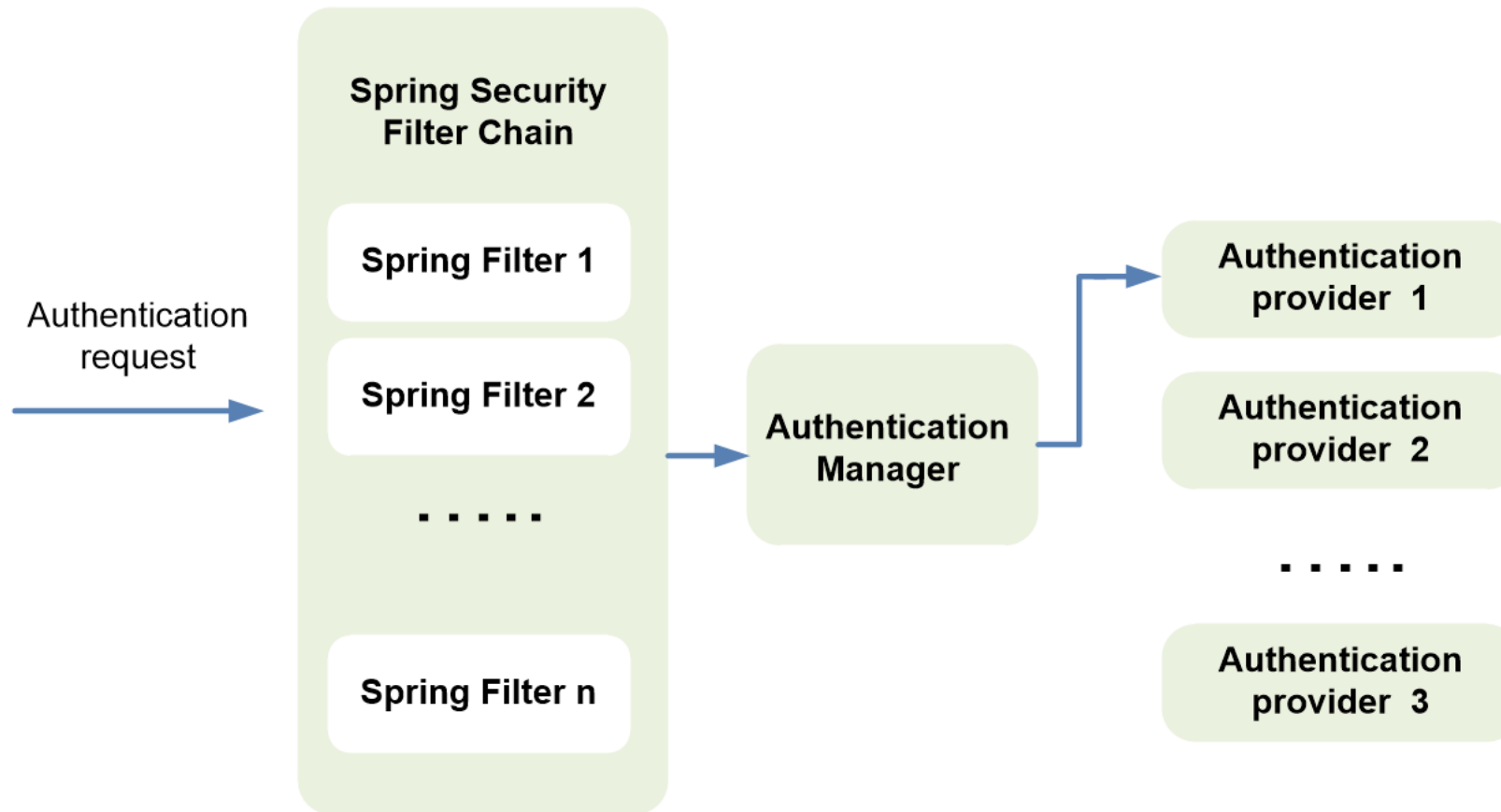


A metamodel of the classes and interfaces involved in the Authentication and Authorization through JPA /DB

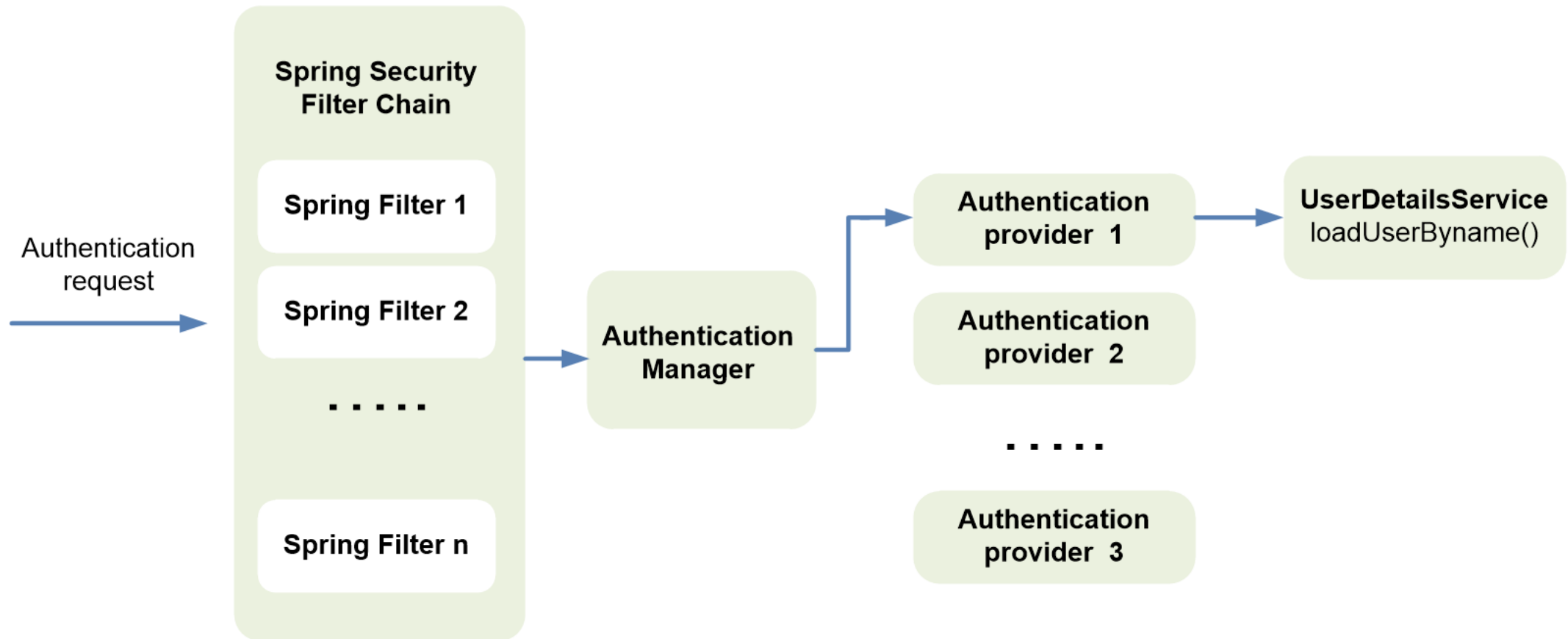
SB Security auth/auth through JPA/DB



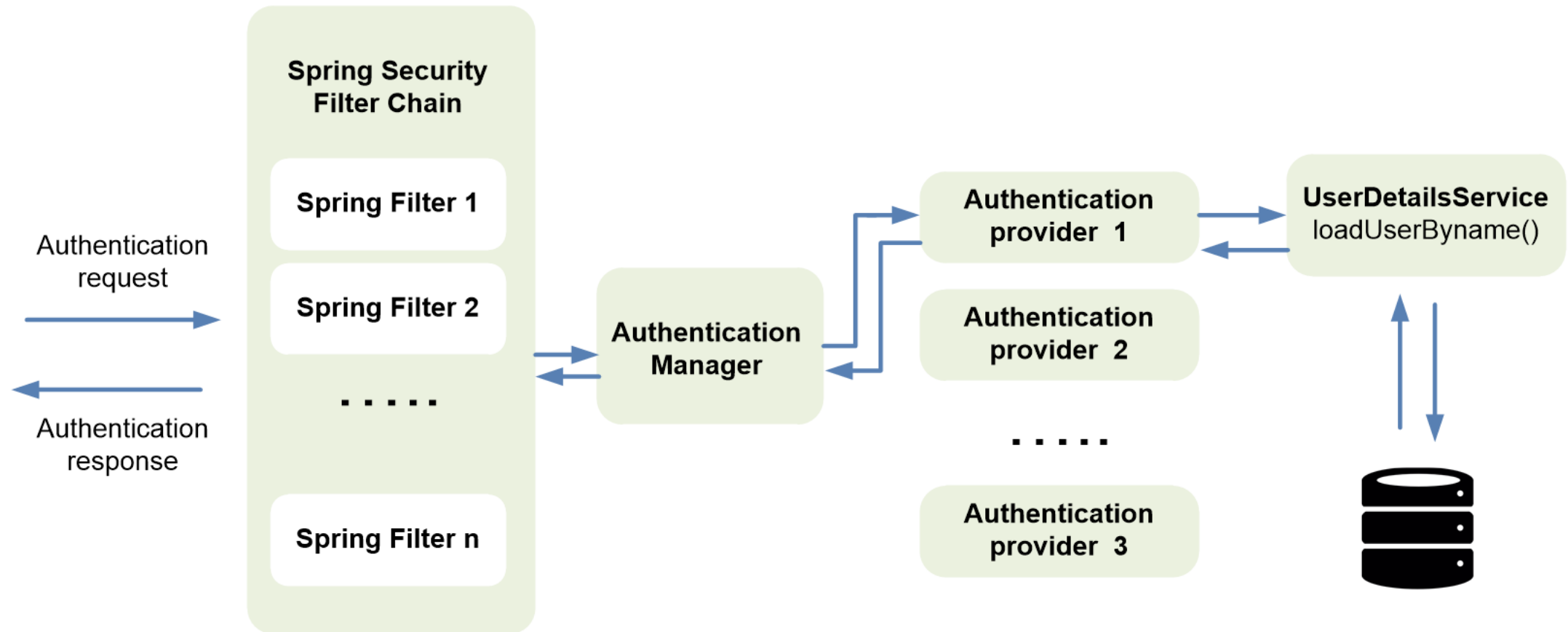
SB Security auth/auth through JPA/DB



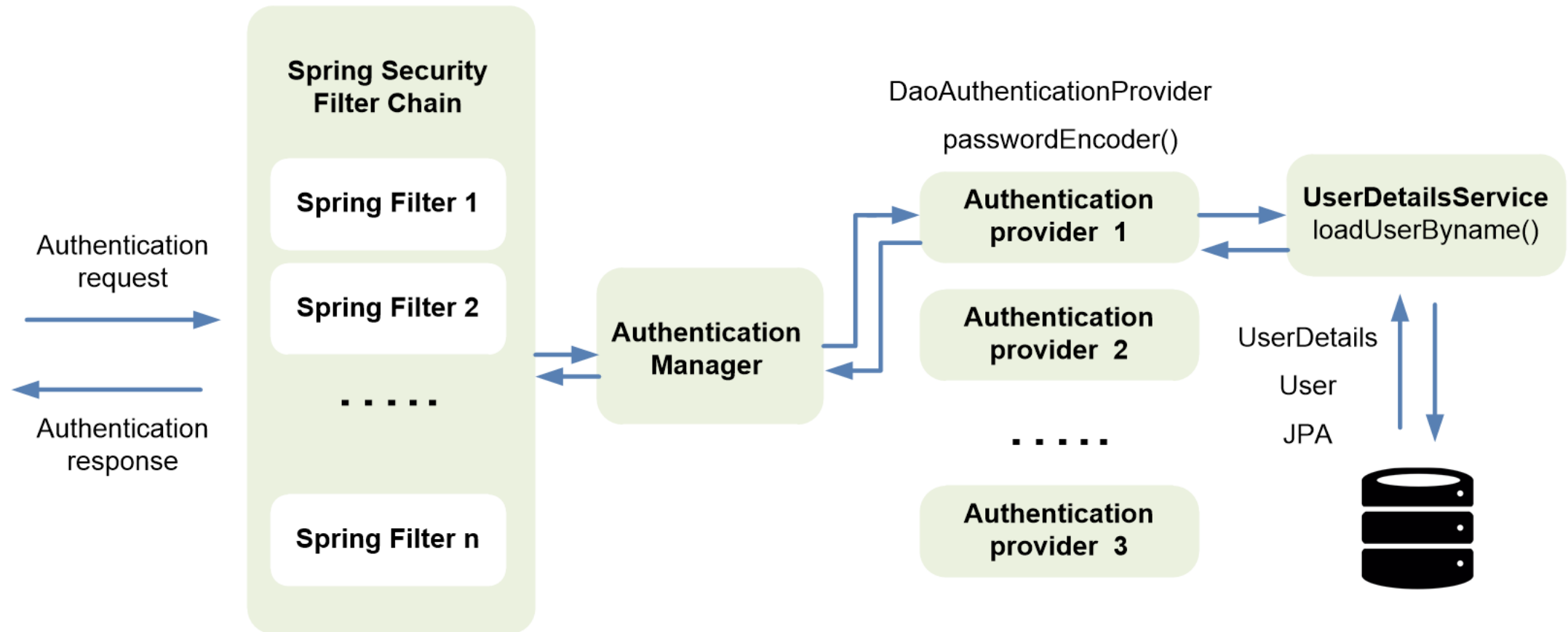
SB Security auth/auth through JPA/DB



SB Security auth/auth through JPA/DB



SB Security auth/auth through JPA/DB



SB Security auth/auth through JPA/DB

```
// userController.java
@RestController
@RequestMapping("/api")
public class SecController {
    @Autowired
    private UserService userService;

    @PostMapping("/new")
    public String addNewUser(@RequestBody User user) {
        return userService.addUser(user);
    }

    @GetMapping("/public")
    public String publicAPI() {
        return "This is an unprotected endpoint";
    }

    @GetMapping("/admin")
    public String adminAPI() {
        return "Protected endpoint - only admins are allowed";
    }

    @GetMapping("/user")
    public String userAPI() {
        return "Protected endpoint - only users are allowed";
    }
}
```

SB Security auth/auth through JPA/DB

```
// userController.java
@RestController
@RequestMapping("/api")
public class UserController {

    // User.java
    public class User {
        @Id
        @GeneratedValue(strategy = GenerationType.AUTO)
        @Column(name = "user_id")
        private UUID id;
        private String name;
        private String password;
        private String roles;
    }

    @GetMapping("/admin")
    public String adminAPI() {
        return "Protected endpoint - only admins are allowed";
    }

    @GetMapping("/user")
    public String userAPI() {
        return "Protected endpoint - only users are allowed";
    }
}
```

SB Security auth/auth through JPA/DB

```
// userController.java
@RestController
@RequestMapping("/api/users")
public class UserController {
    @Autowired
    private UserService userService;

    @PostMapping
    public String addUser(@RequestBody User user) {
        return userService.addUser(user);
    }
}

// User.java
public class User {
    private String username;
    private String password;

    // getters and setters
}

// UserService.java
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;

    public String addUser(User user) {
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        userRepository.save(user);
        log.info("A new user {} is added", user.toString());
        return "A new user is added to system";
    }
}
```

SB Security auth/auth through JPA/DB

```
// userController.java
```

```
@RestController  
@RequestMapping("/api")  
public class UserController {  
    // User.java
```

```
    // UserService.java
```

```
    // UserRepository.java
```

```
    public interface UserRepository extends JpaRepository<User, UUID> {  
        Optional<User> findByName(String username);  
    }
```

```
    private PasswordEncoder passwordEncoder;
```

```
    public String addUser(User user) {  
        user.setPassword(passwordEncoder.encode(user.getPassword()));  
        userRepository.save(user);  
        log.info("A new user {} is added", user.toString());  
        return "A new user is added to system";  
    }
```

```
    @GetMapping("/api/users")  
    public List<User> getAllUsers() {  
        return userRepository.findAll();  
    }  
}
```

SB Security auth/auth through JPA/DB

```
// SecurityConfig
...
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
    @Bean
    public UserDetailsService userDetailsService(PasswordEncoder encoder) {
        ...
        .requestMatchers("/api/user").hasAnyRole("USER")
        .requestMatchers("/api/user").hasAuthority("USER")
        ...
        return new InMemoryUserDetailsManager(admin, user);
        return new MyUserDetailsService();
    }
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        ..
    }
    ..
}
```

SB Security auth/auth through JPA/DB

```
// SecurityConfig
...
@Bean
public AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authenticationProvider=new DaoAuthenticationProvider();
    authenticationProvider.setUserDetailsService(userDetailsService());
    authenticationProvider.setPasswordEncoder(passwordEncoder());
    return authenticationProvider;
}
}
```

DaoAuthenticationProvider is an AuthenticationProvider implementation that uses a UserDetailsService and PasswordEncoder to authenticate a username and password.

SB Security auth/auth through JPA/DB

```
// MyUserDetails
...
public class MyUserDetails extends User implements UserDetails {
    private User user;
    public MyUserDetails(final User user) {this.user = user;}
    @Override
    public List<? extends GrantedAuthority> getAuthorities() {
        SimpleGrantedAuthority authority = new SimpleGrantedAuthority(user.getRoles());
        return Arrays.asList(authority);}
    @Override
    public String getPassword() {
        return user.getPassword();}
    @Override
    public String getUsername() {
        return user.getName();}
    //hard-coding these attributes
    @Override
    public boolean isAccountNonExpired() {
        return true;}
    ..
}
```

SB Security auth/auth through JPA/DB

```
// MyUserDetailsService
...
@Component
public class MyUserDetailsService implements UserDetailsService {

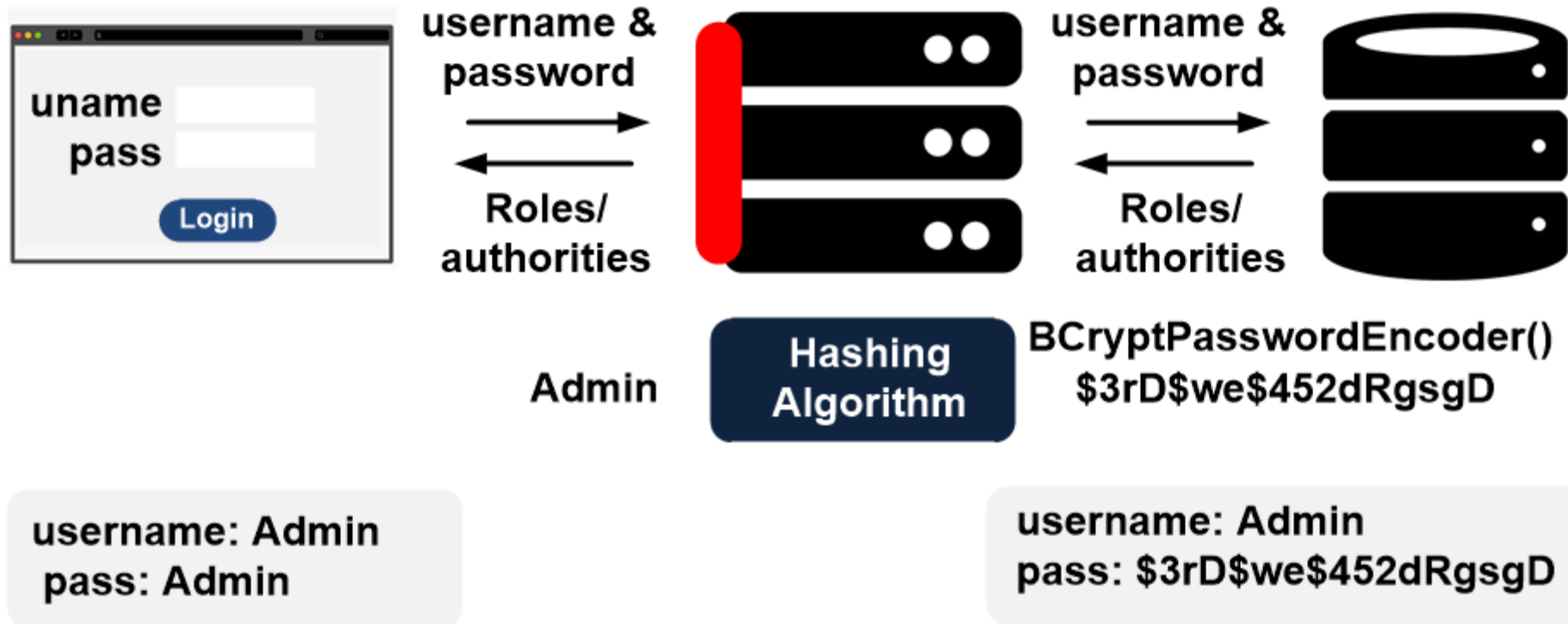
    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException{
        Optional<User> user = userRepository.findByName(username);
        user
            .orElseThrow(() -> new UsernameNotFoundException(username + "not found"));
        return user.map(MyUserDetails::new).get();
    }
}
```

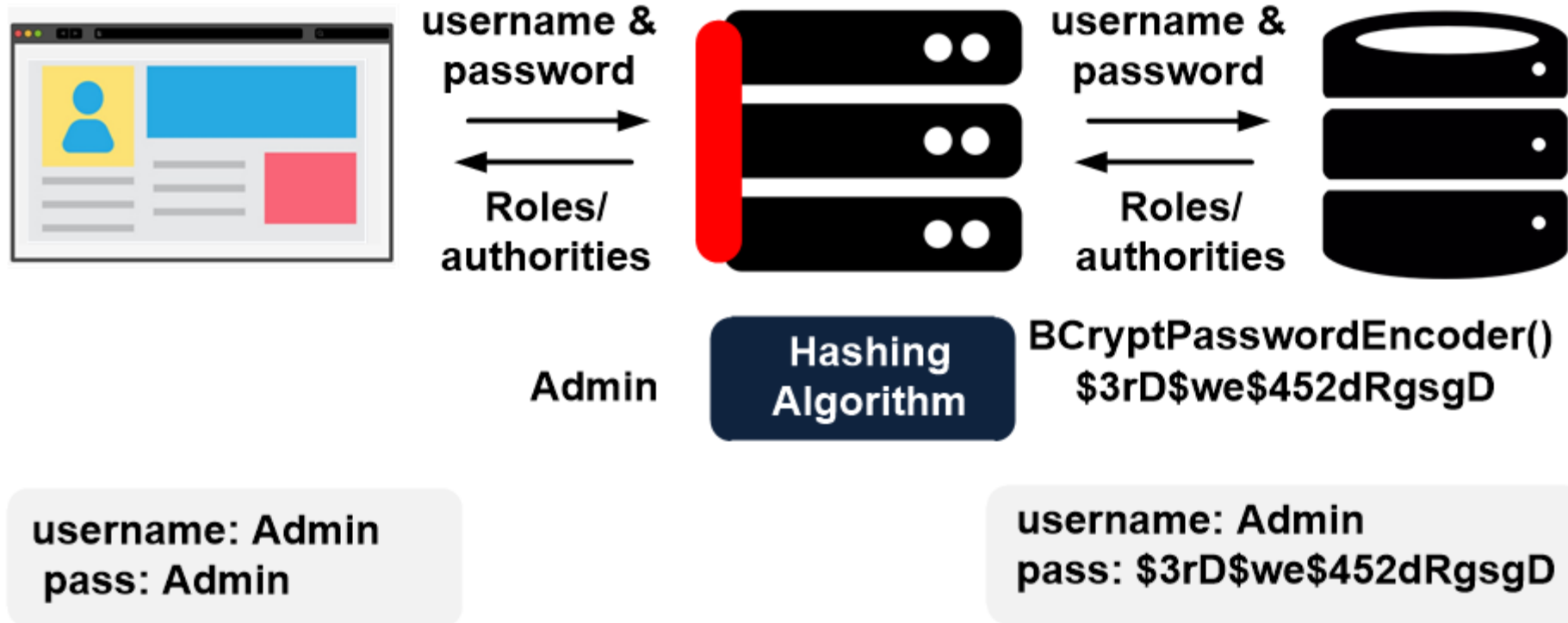

What is wrong with this system?

Try it yourself ...

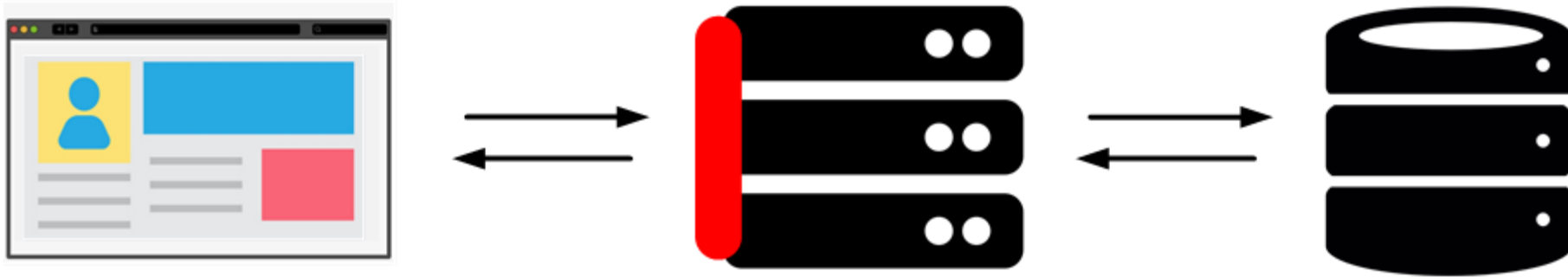
What is wrong with this system?



What is wrong with this system?

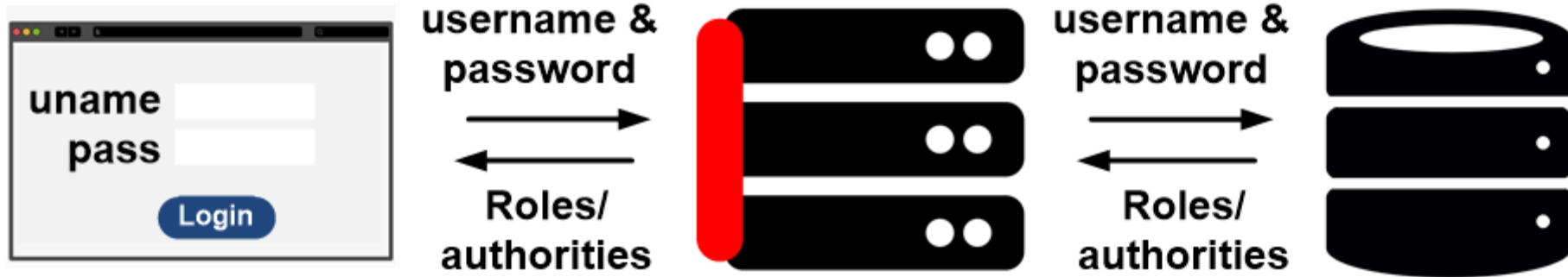


What is wrong with this system?



I want to navigate to another protected directory!

What is wrong with this system?

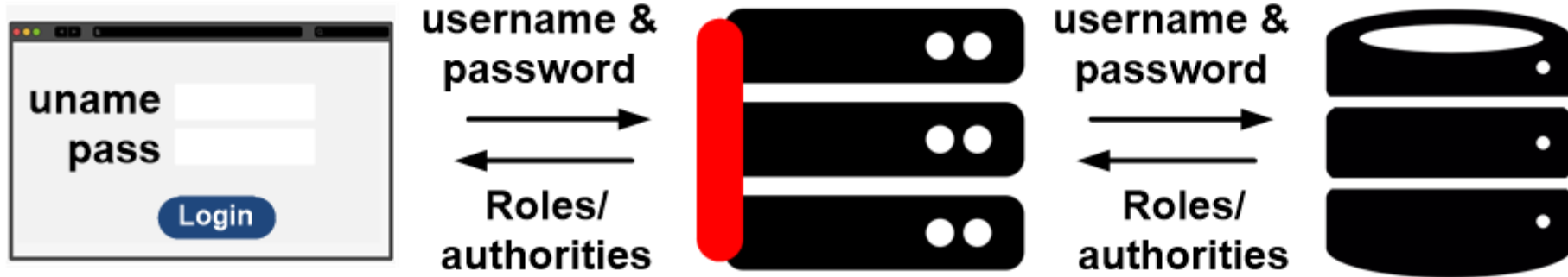


What is wrong with this system?

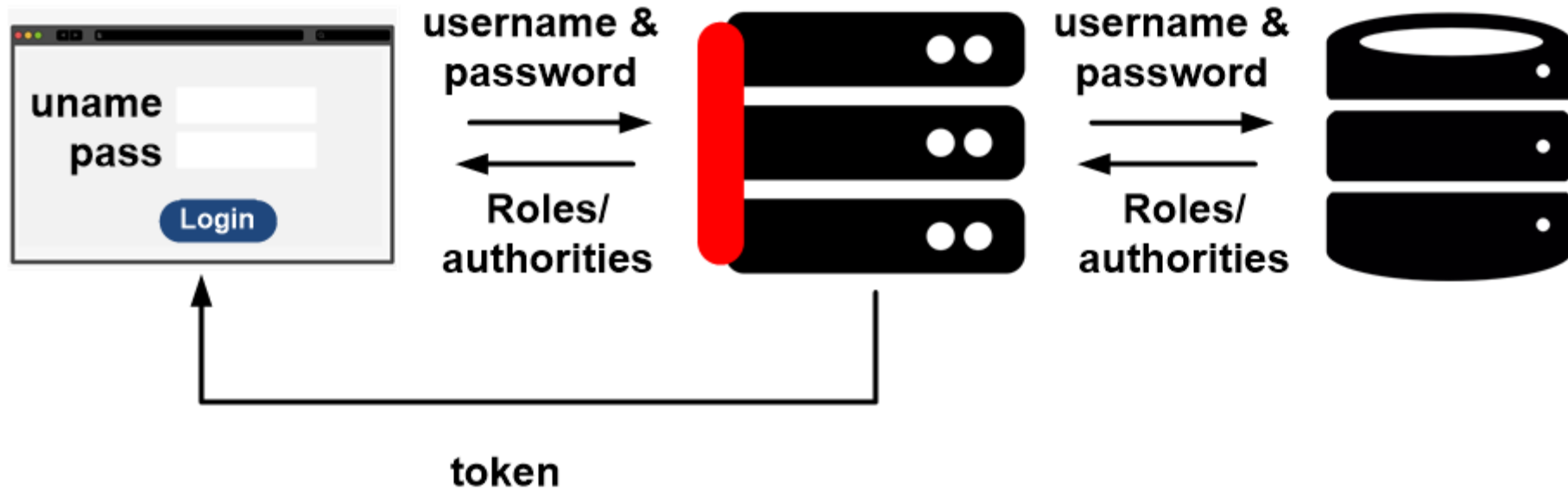


We need a solution!

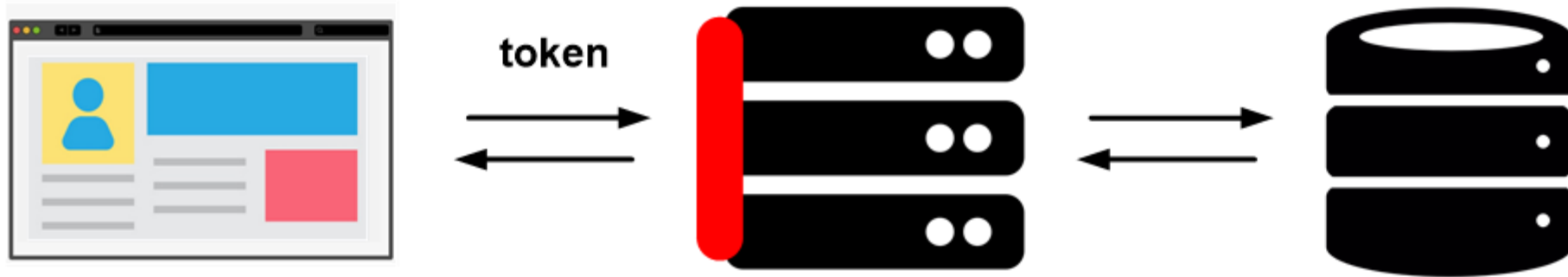
What is wrong with this system?



What is wrong with this system?



What is wrong with this system?





Cookies

Cookies

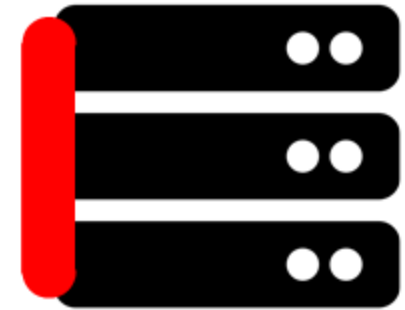
A **cookie** is a **small bit** of information that a website stores within your browser.

Cookies themselves are **not harmful**. However, some **cyber attacks** can hijack cookies and enable access to your browsing sessions.

The danger lies in their ability to track individuals' browsing histories.



Client



Server

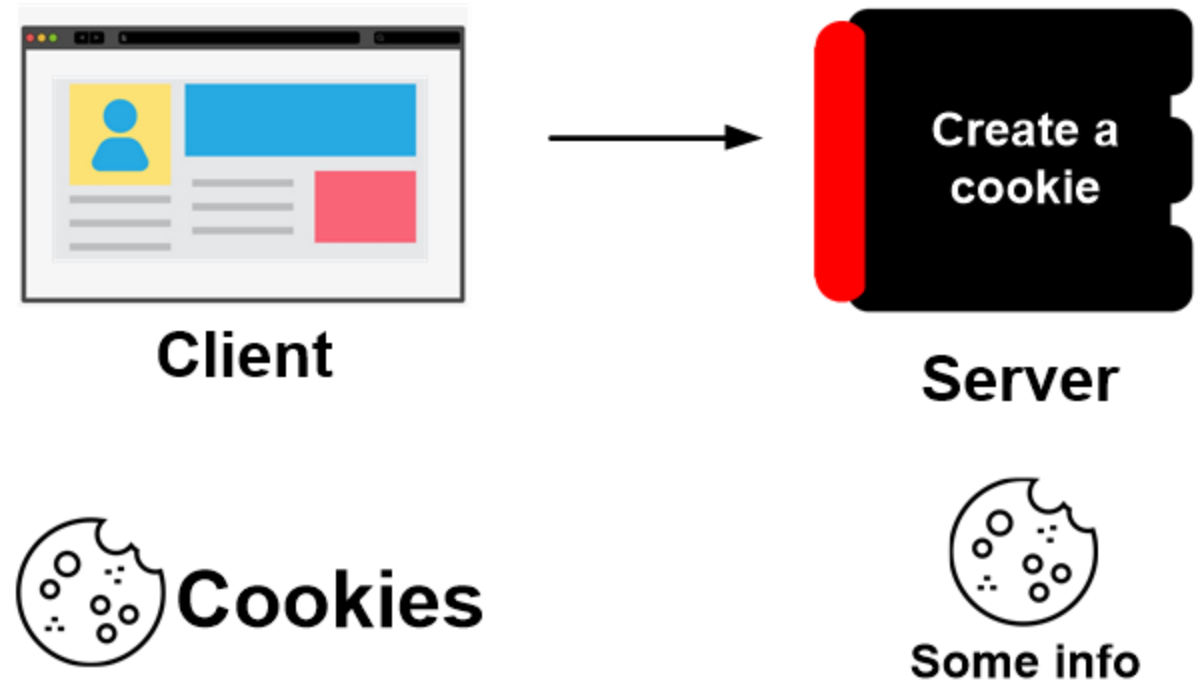


Cookies

Cookies

A cookie is created when you visit a website.

A **cookie** can be “session” or (non-persistent **cookie**), or persistent **cookie** .

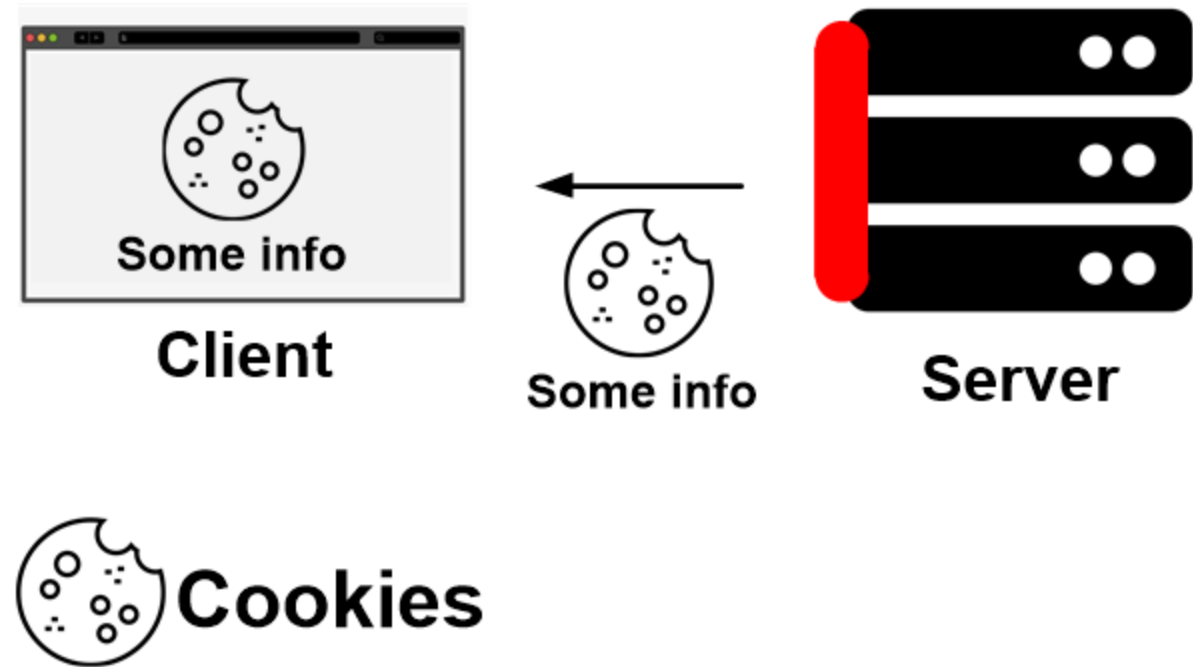


Cookies

A cookie is created when you visit a website.

A **cookie** can be “session” or (non-persistent **cookie**), or persistent **cookie** .

After its creation, it is sent to be stored in the user browser.



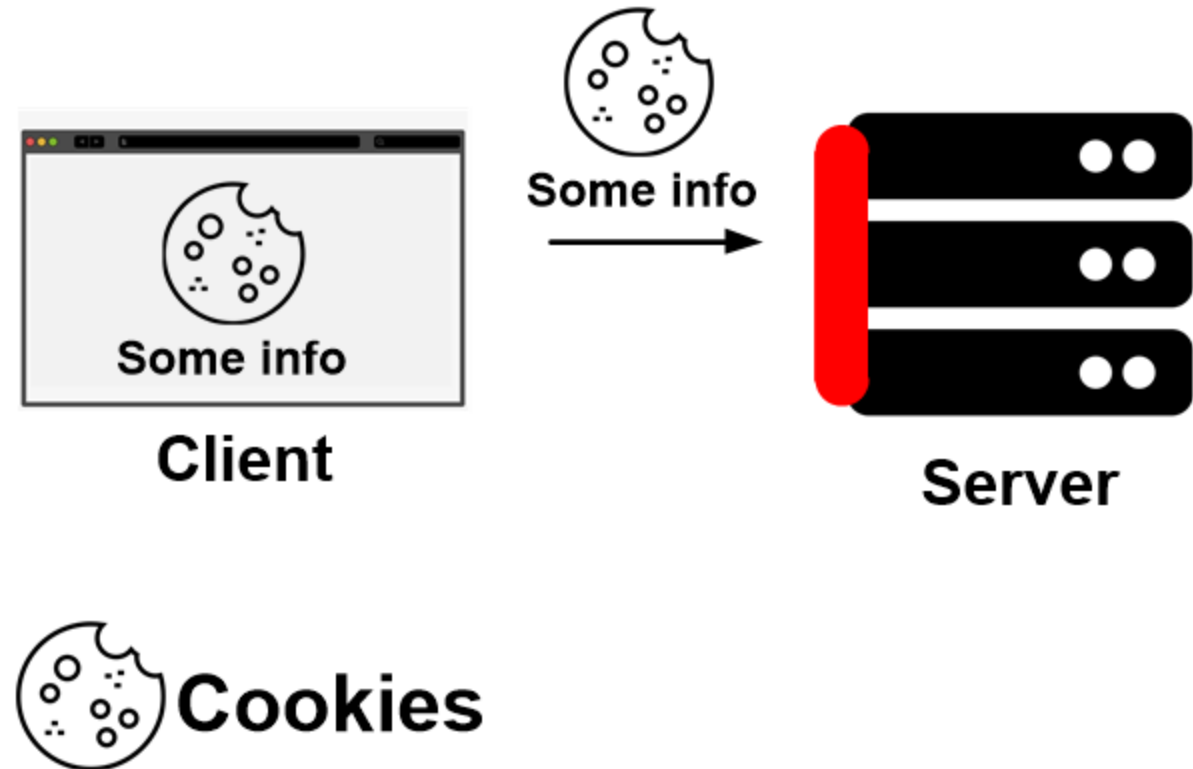
Cookies

A cookie is created when you visit a website.

A **cookie** can be “session” or (non-persistent **cookie**), or persistent **cookie** .

After its creation, it is sent to be stored in the user browser.

When you revisit the website, your browser sends the information stored in the cookie **back** to the site.



JSON Web Token (JWT)

JSON Web Token (JWT)

JSON Web Token (JWT), pronounced "jot", is an open industry standard ([RFC 7519](#)) for representing claims “securely” between two parties.



JWT defines a compact and **self-contained** way for “securely” transmitting information between parties **as a JSON object**.

Note: **JWT** is only a one way for authenticating users.

JSON Web Token (JWT)

JWT is self-contained.

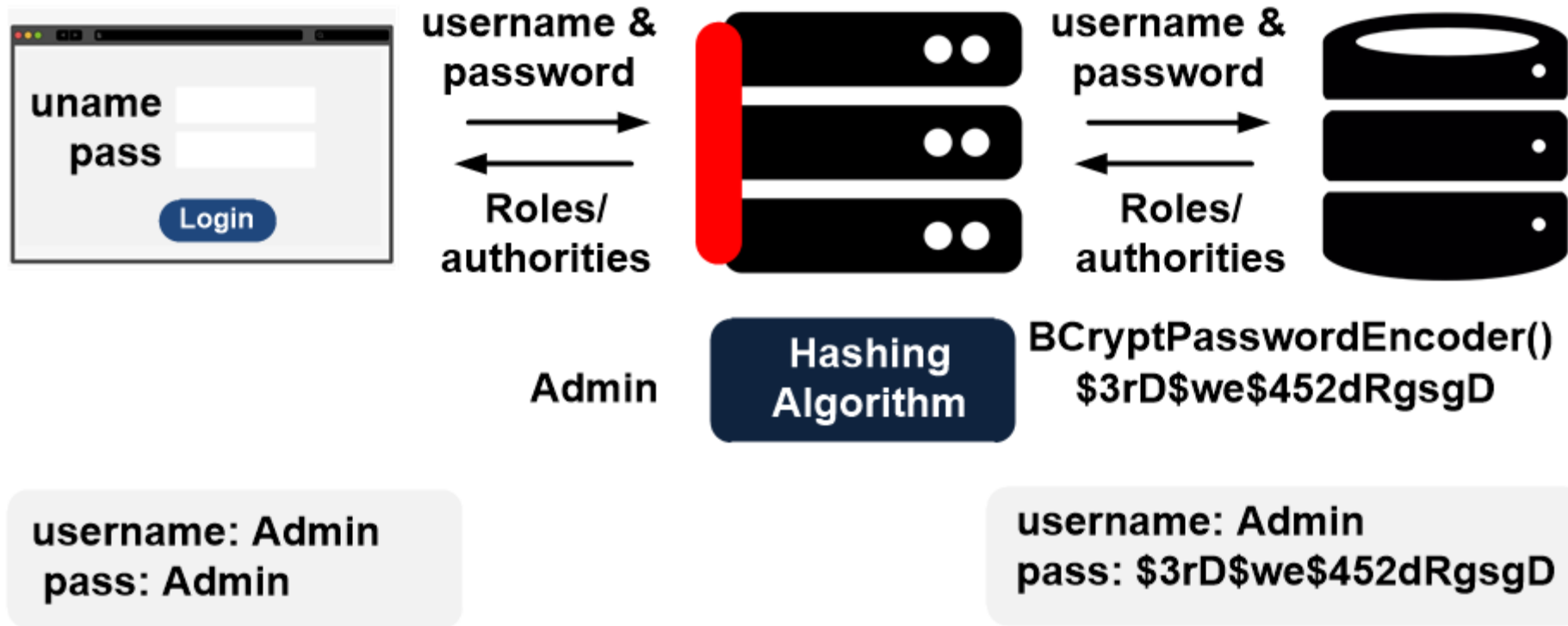
- The auth data is stored in the token not on the server.
- The client can read the data, but cannot change it (`httpOnly: true`).
- The server "signs" the data by hashing it with a secret.
- The server can verify the token.



JWT is (somehow) secure way to exchange authentication information.

JWT can be subject to some security attacks (e.g., Cross Site Scripting (XSS)).

How JWT is created, signed and verified



I need to know who is this user!

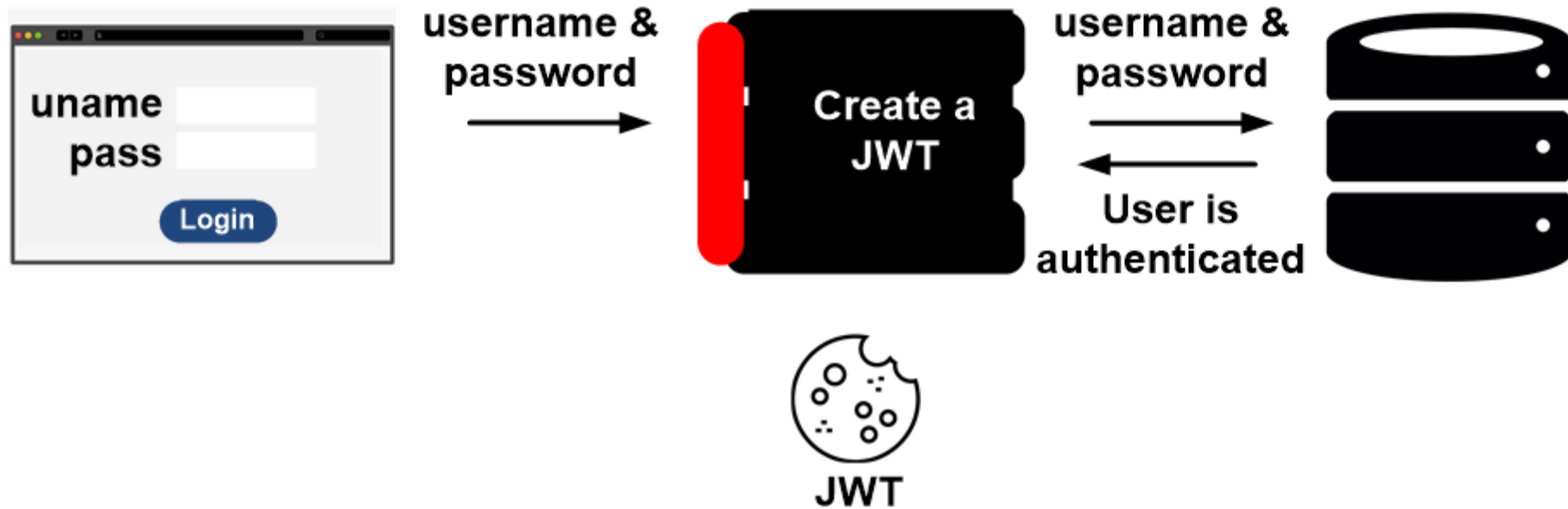
In our system, I need to have the user's information in my system, to **authenticate** him/her.

How JWT is created, signed and verified



A user can be **authenticated** by **checking** the information he/she provide with the information stored in the system.

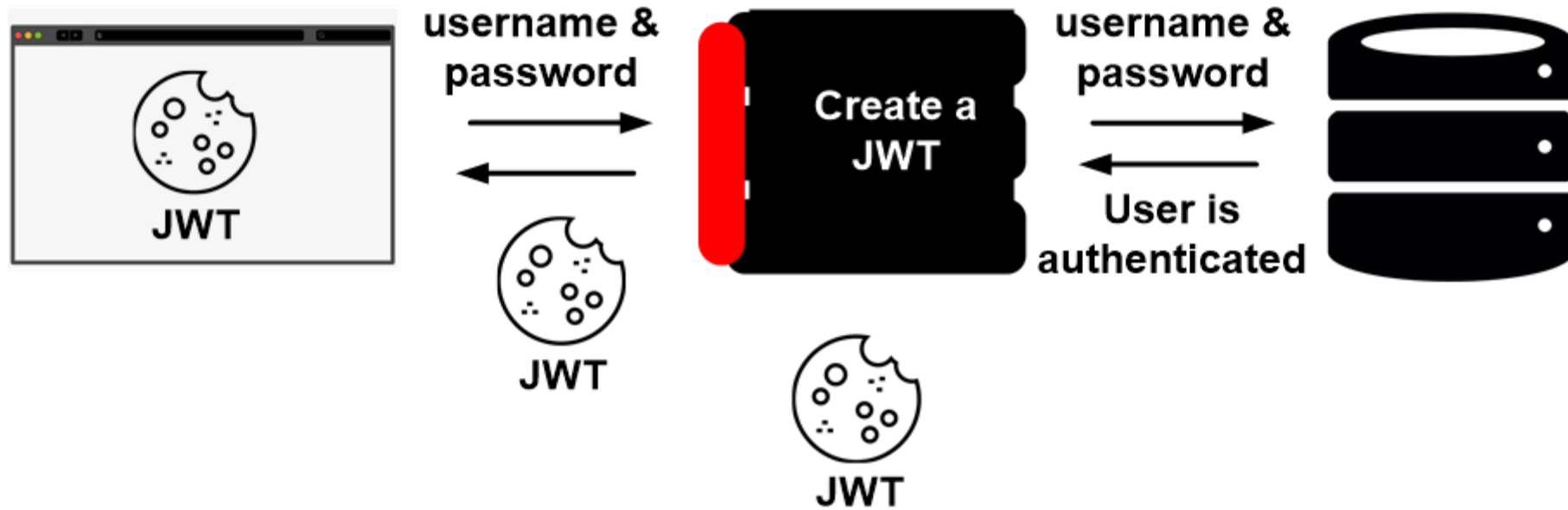
How JWT is created, signed and verified



A user can be **authenticated** by **checking** the information he/she provide with the information stored in the system.

If we have a **match** (the user is **authenticated**), the system will **create the JWT**.

How JWT is created, signed and verified



After creating the JWT, it is sent to be stored in the user's browser.

How JWT is created, signed and verified



Result is OK proceed to resource
Result is not OK, redirect to login/signup page

After creating the JWT, it is sent to be stored in the user's browser.

When the user **revisit** the website, the browser automatically sends the JWT (and any cookies) to the **server**.

JWT structure

JWT structure



Header

■

Payload

■

Signature

JWT structure



Header

Payload

Signature

Header

consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.

Payload

contains the claims, which are statements about the user and additional data.

Signature

makes the token secure by taking the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

JWT structure



Header

Payload

Signature

Header

consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.

```
{ "alg": "HS256",  
  "typ": "JWT" }
```

Payload

contains the claims, which are statements about the user and additional data.

```
{ "sub": 1234567890,  
  "name": "John Doe",  
  "admin": true }
```

Signature

makes the token secure by taking the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

```
HMACSHA256(  
  base64UrlEncode(header) + "." +  
  base64UrlEncode(payload), secret)
```

JWT structure



Header

Payload

Signature

Header

consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.

Payload

contains the claims, which are statements about the user and additional data.

Signature

makes the token secure by taking the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

```
{ "alg": "HS256",  
  "typ": "JWT" }
```

encoded by
Base64Url

```
{ "sub": 1234567890,  
  "name": "John Doe",  
  "admin": true }
```

encoded by
Base64Url

HMACSHA256(
 base64UrlEncode(**header**) + "." +
 base64UrlEncode(**payload**), **secret**)

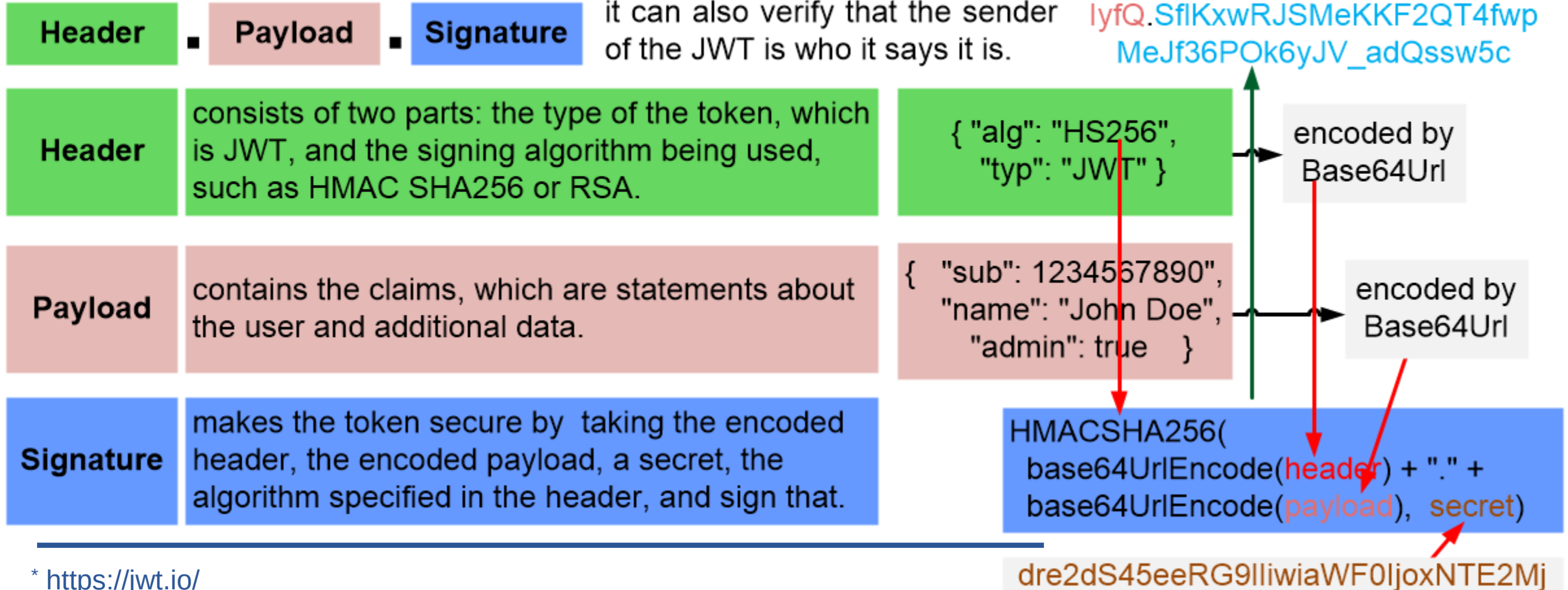
dre2dS45eeRG9IliwiaWF0ljoXNTE2Mj

JWT structure



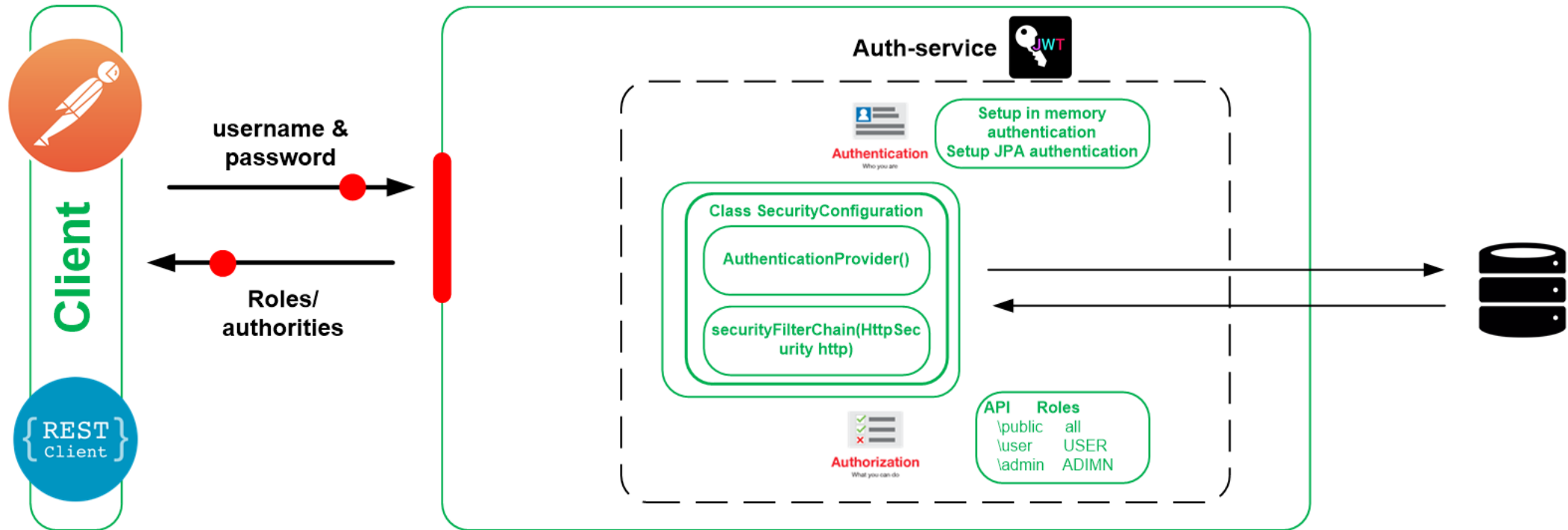
The signature is used to verify that the message was not changed, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

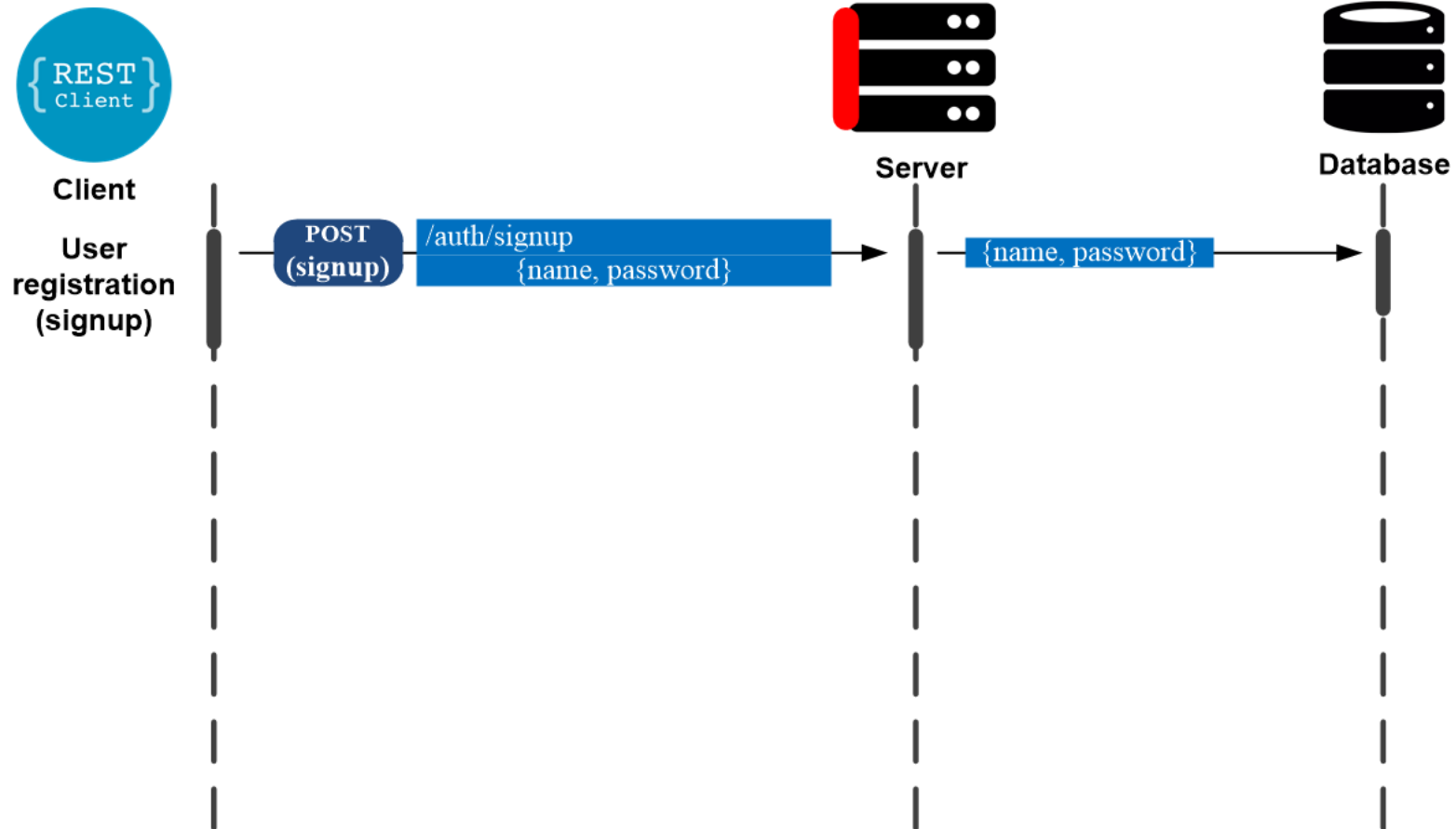


Building an Auth Service for a Spring Boot App - backend

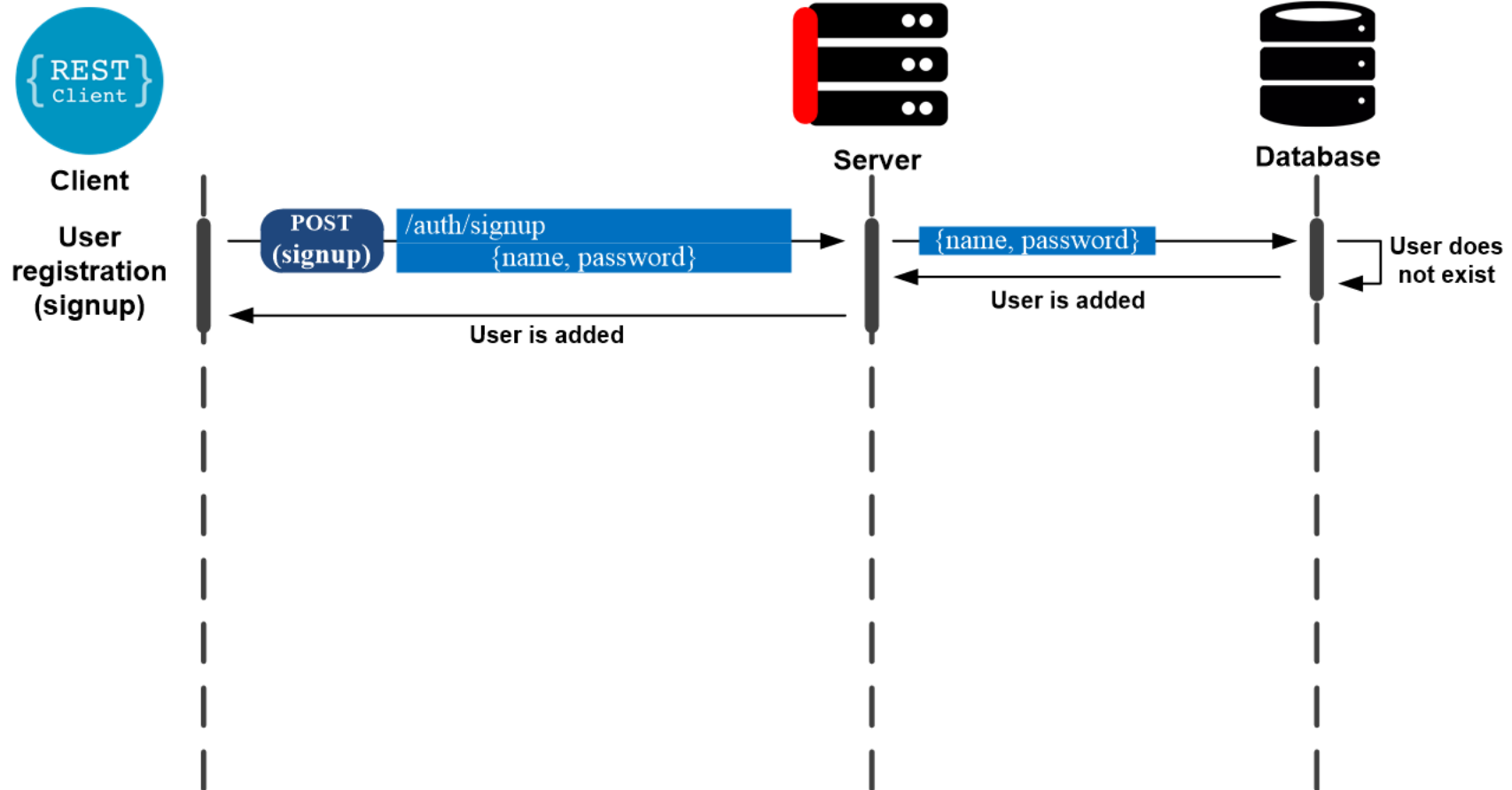
An Auth Service for a Spring Boot App



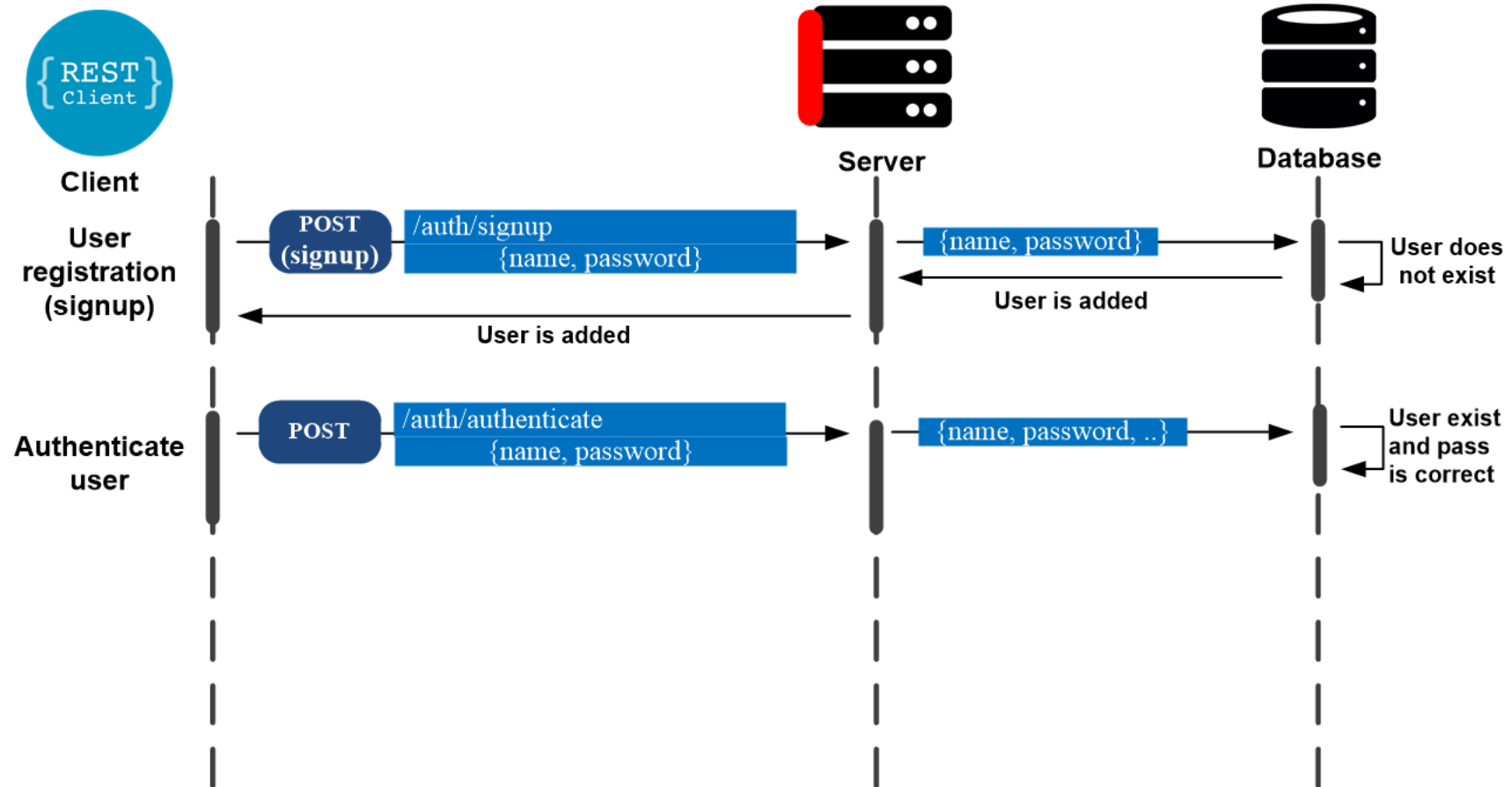
An Auth Service for a Spring Boot App - WF



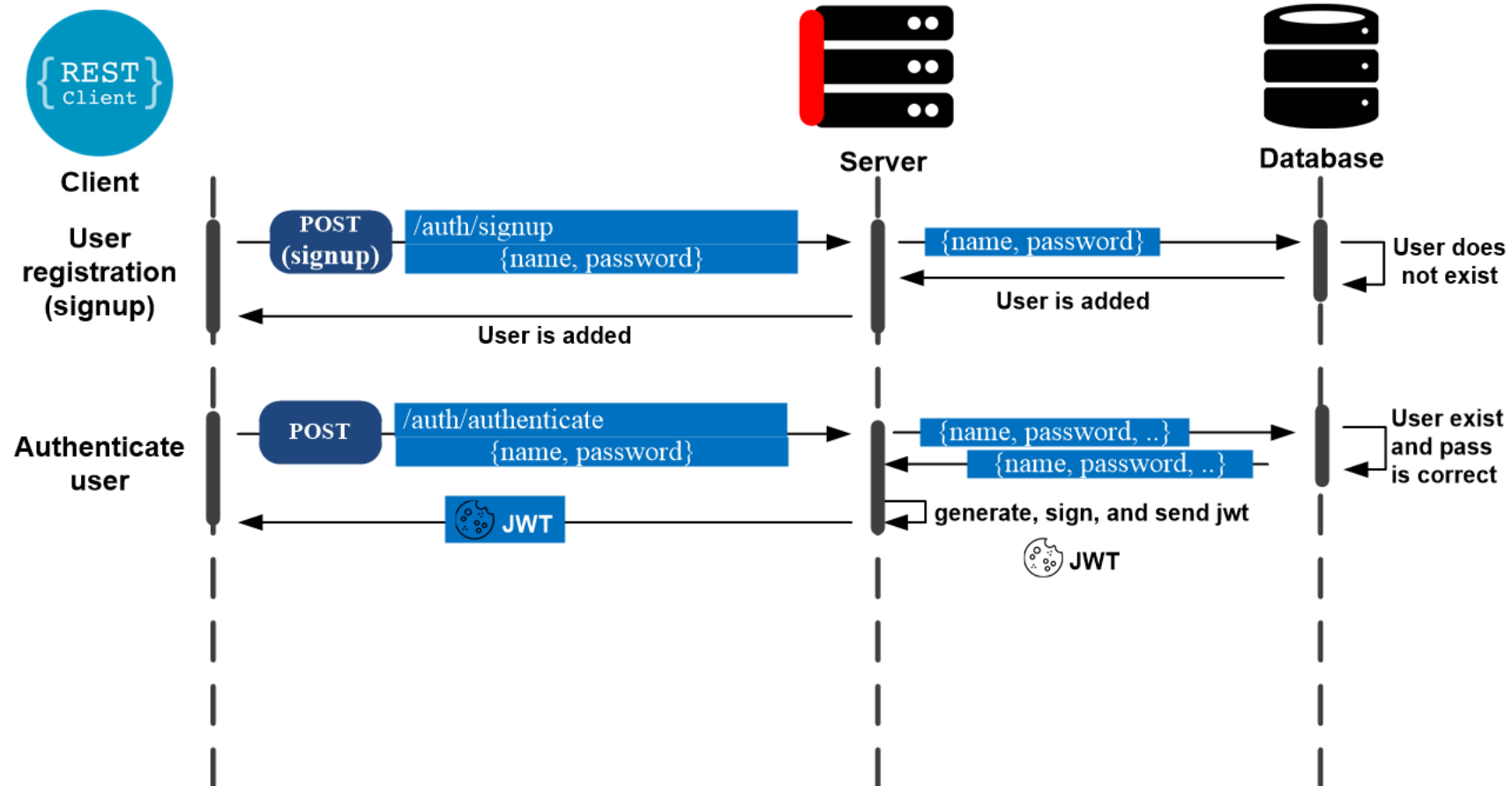
An Auth Service for a Spring Boot App - WF



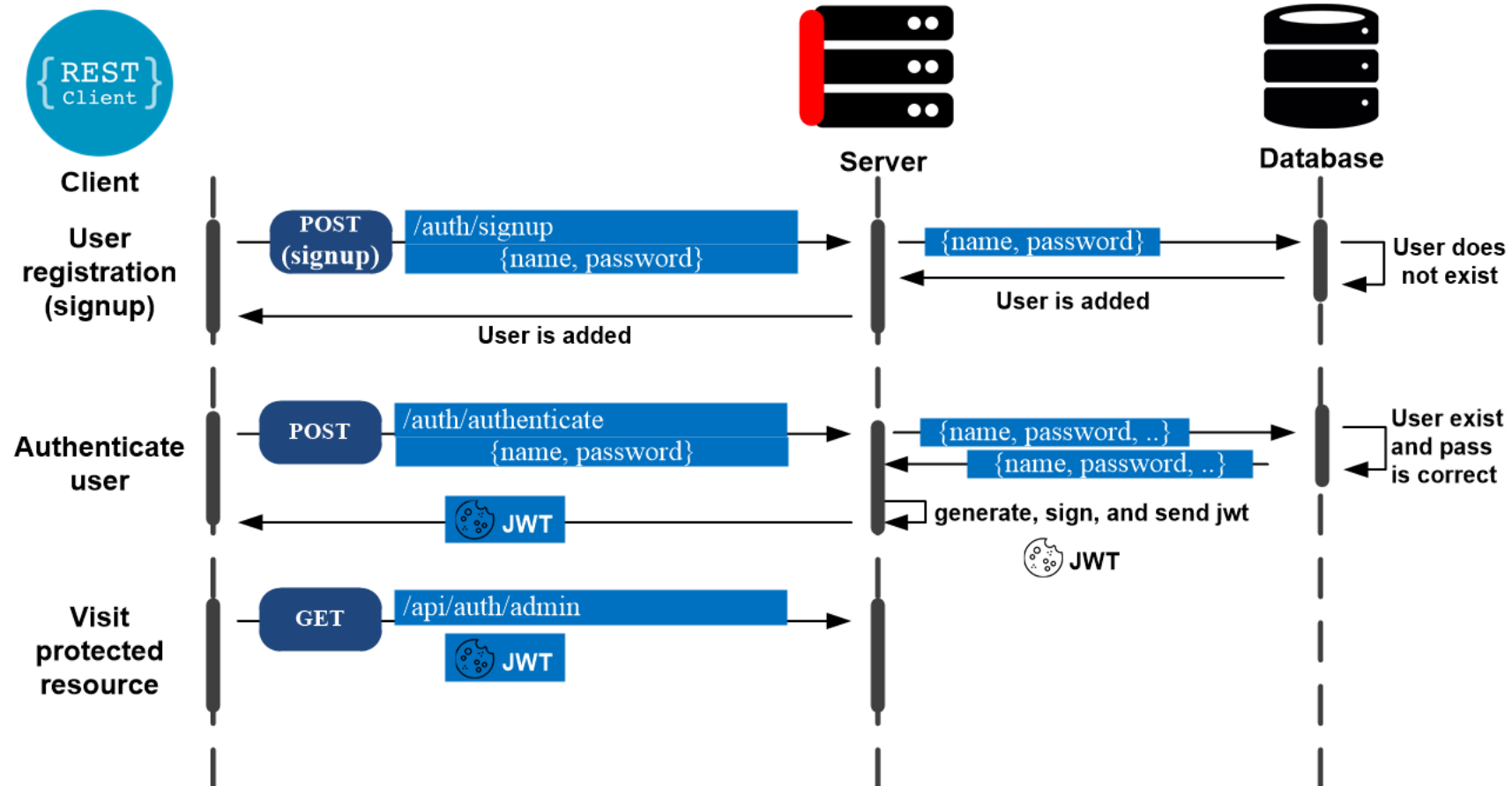
An Auth Service for a Spring Boot App - WF



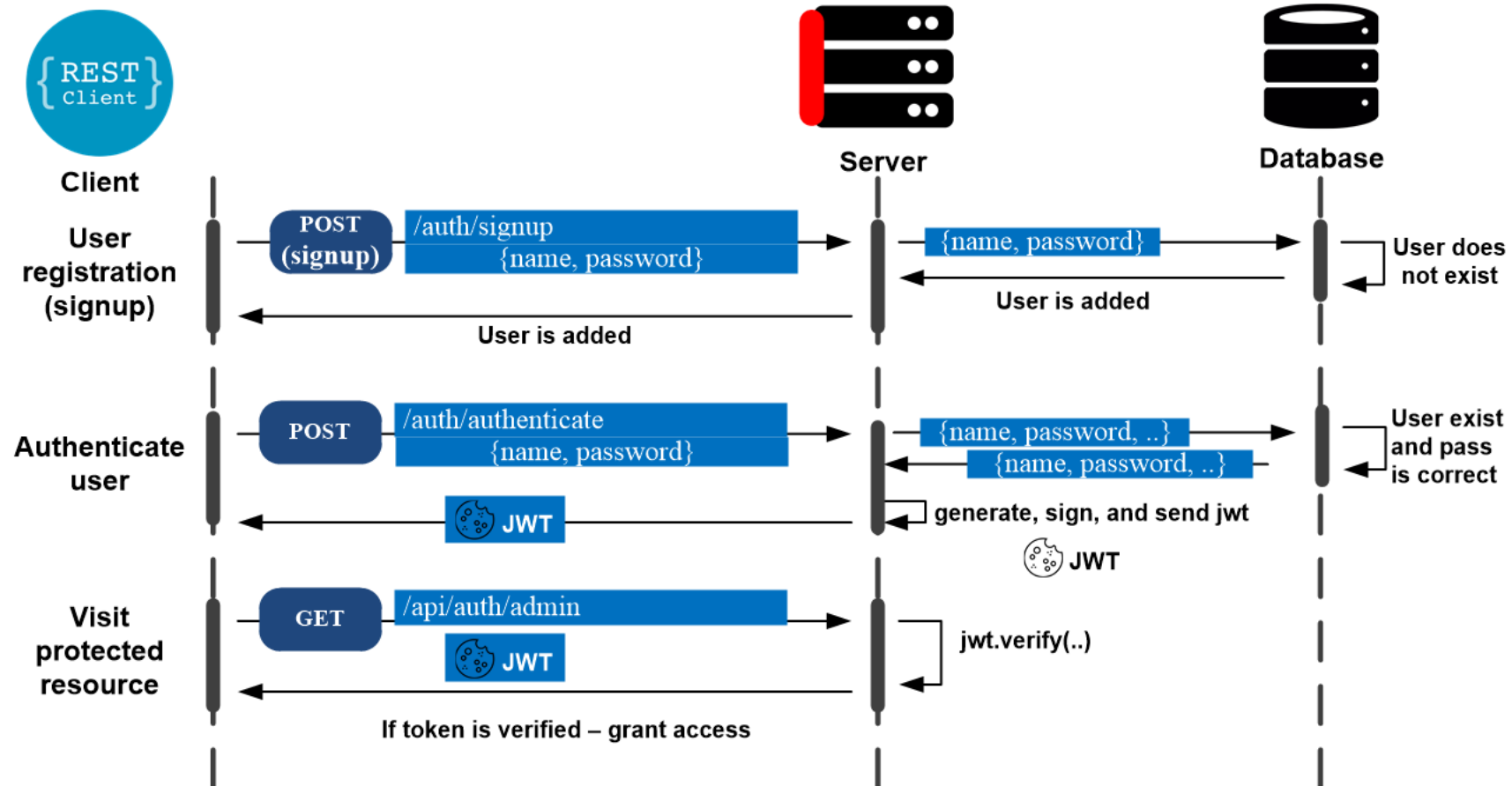
An Auth Service for a Spring Boot App - WF



An Auth Service for a Spring Boot App - WF



An Auth Service for a Spring Boot App - WF



An Auth Service for a Spring Boot App

An Auth Service for a Spring Boot App

```
auth-service-jwt
├── config
│   ├── MyUserDetails.java
│   ├── MyUserDetailsService.java
│   └── SecurityConfig.java
├── jwt
│   ├── JwtAuthFilter.java
│   └── JwtService.java
├── users
│   ├── controller
│   │   └── userController.java
│   ├── dto
│   │   └── UserDto.java
│   ├── model
│   │   └── User.java
│   ├── repository
│   │   └── userRepository.java
│   └── service
│       └── userService.java
```

An Auth Service for a Spring Boot App

auth-service-jwt

```
└─ config
    └─ MyUserDetails.java
    └─ MyUserDetailsService.java
    └─ SecurityConfig.java
└─ jwt
    └─ JwtAuthFilter.java
    └─ JwtService.java
└─ users
    └─ controller
        └─ userController.java
    └─ dto
        └─ UserDto.java
    └─ model
        └─ User.java
    └─ repository
        └─ userRepository.java
    └─ service
        └─ userService.java
```

With respect to no JWT:

- MyUserDetails.java, MyUserDetailsService.java, User.java, userRepository.java, and userService.java are the same, i.e., **have not been modified.**

An Auth Service for a Spring Boot App

auth-service-jwt

```
└─ config
    └─ MyUserDetails.java
    └─ MyUserDetailsService.java
    └─ SecurityConfig.java
└─ jwt
    └─ JwtAuthFilter.java
    └─ JwtService.java
└─ users
    └─ controller
        └─ userController.java
    └─ dto
        └─ UserDto.java
    └─ model
        └─ User.java
    └─ repository
        └─ userRepository.java
    └─ service
        └─ userService.java
```

With respect to no JWT:

- MyUserDetails.java, MyUserDetailsService.java, User.java, userRepository.java, and userService.java are the same, i.e., **have not been modified.**
- SecurityConfig.java and userController.java **have been modified.**

An Auth Service for a Spring Boot App

auth-service-jwt

```
├── config
│   ├── MyUserDetails.java
│   ├── MyUserDetailsService.java
│   └── SecurityConfig.java
├── jwt
│   ├── JwtAuthFilter.java
│   └── JwtService.java
├── users
│   ├── controller
│   │   └── userController.java
│   ├── dto
│   │   └── UserDto.java
│   ├── model
│   │   └── User.java
│   ├── repository
│   │   └── userRepository.java
│   └── service
│       └── userService.java
```

With respect to no JWT:

- MyUserDetails.java, MyUserDetailsService.java, User.java, userRepository.java, and userService.java are the same, i.e., **have not been modified**.
- SecurityConfig.java and userController.java **have been modified**.
- JwtAuthFilter.java, JwtService.java, and UserDto.java **are new**.

SecurityConfig.java

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .cors(withDefaults())
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/api/auth/public", "/api/auth/signup",
                "/api/auth/authenticate").permitAll()
            .requestMatchers("/api/auth/admin").hasAuthority("ADMIN")
            .requestMatchers("/api/auth/user").hasAuthority("USER")
            .anyRequest().authenticated())
        .sessionManagement()
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        .and()
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

SecurityConfig.java

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        // You need this to deactivate the cors issue
        @Bean
        public CorsConfigurationSource corsConfigurationSource() {
            CorsConfiguration configuration = new CorsConfiguration();
            configuration.setAllowedOrigins(Arrays.asList("http://localhost:8080"));
            //or add * to allow all origins
            configuration.setAllowCredentials(true);
            configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH",
                "DELETE", "OPTIONS")); //to set allowed http methods
            configuration.setAllowedHeaders(Arrays.asList("Authorization", "Cache-Control",
                "Content-Type"));
            configuration.setExposedHeaders(Arrays.asList("custom-header1", "custom-
                header2"));
            UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
            source.registerCorsConfiguration("/**", configuration);
            return source;
        }
}
```

UserController.java

```
..
//This endpoint will receive the userDto, authenticate her/him with existing users in
the database, then if authenticated, it will create the jwt
@PostMapping("/authenticate")
public String authenticateAndGetToken(@RequestBody UserDto userDto) {

    Authentication authentication = authenticationManager.authenticate(new
    UsernamePasswordAuthenticationToken(userDto.getName(), userDto.getPassword()));
    // If the user is authenticated we generate the token, otherwise, we throw an
    exception
    if (authentication.isAuthenticated()) {
        jwtService.generateToken(userDto.getName()).toString();
        return jwtService.generateToken(userDto.getName());
    } else {
        throw new UsernameNotFoundException("The user cannot be authenticated!");
    }
}
```

JwtAuthFilter.java

```
public class JwtAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain) throws ServletException, IOException {
        String authHeader = request.getHeader("Authorization");
        String token = null;
        String username = null;
        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            token = authHeader.substring(7);
            username = jwtService.extractUsername(token);
        }
        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userService.loadUserByUsername(username);
            if (jwtService.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePassword
AuthenticationToken(userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
            }
            filterChain.doFilter(request, response);
        }
    }
}
```

JwtService.java

```
public class JwtService {  
    ..  
    public String generateToken(String userName) {  
        Map<String, Object> claims = new HashMap<>();  
        return createToken(claims, userName);  
    }  
    private String createToken(Map<String, Object> claims, String userName) {  
        UserDetails userDetails = userService.loadUserByUsername(userName);  
        Collection<? extends GrantedAuthority> roles = userDetails.getAuthorities();  
        String rolesClaim = roles.toString();  
        int start = rolesClaim.indexOf("[");  
        int end = rolesClaim.indexOf("]");  
        rolesClaim = rolesClaim.substring(start + 1, end);  
        claims.put("roles", rolesClaim);  
        return Jwts.builder()  
            .setClaims(claims) // setting our claims  
            .setSubject(userName)  
            .setIssuedAt(new Date(System.currentTimeMillis()))  
            .setExpiration(new Date(System.currentTimeMillis()+1000*60*60))  
            .signWith(signingKey(), SignatureAlgorithm.HS256).compact();  
    }  
}
```

JwtService.java

```
public class JwtService {  
    ..  
    public String generateToken(String userName) {  
        Map<String, Object> claims = new HashMap<>();  
        return createToken(claims, userName);  
    }  
    private String createToken(Map<String, Object> claims, String userName) {  
        UserDetails userDetails = userService.loadUserByUsername(userName);  
        Collection<? extends GrantedAuthority> roles = userDetails.getAuthorities();  
        String rolesClaim = roles.toString();  
        int start = rolesClaim.indexOf("[");  
        int end = rolesClaim.indexOf("]");  
        rolesClaim = rolesClaim.substring(start + 1, end);  
        claims.put("roles", rolesClaim);  
        return Jwts.builder()  
            .setSubject(userName)  
            .setExpiration(Date.from(Instant.now().plusSeconds(3600)))  
            .signWith(signingKey())  
            .compact();  
    }  
    public static final String SECRET =  
        "4D6351665468576D5A7134743777217A25432A462D4A614E645267556B586E32";  
    private Key signingKey() {  
        byte[] keyDecoder = Decoders.BASE64.decode(SECRET);  
        return Keys.hmacShaKeyFor(keyDecoder);  
    }  
}
```

```
public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
    final Claims claims = extractAllClaims(token);
    return claimsResolver.apply(claims);}

private Claims extractAllClaims(String token) {
    return Jwts
        .parserBuilder()
        .setSigningKey(signingKey())
        .build()
        .parseClaimsJws(token)
        .getBody();}

public String extractUsername(String token) {
    return extractClaim(token, Claims::getSubject);}

public String extractRoles(String token) {
    String claimRoles = extractAllClaims(token).get("roles", String.class);
    return claimRoles;}

public Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);}

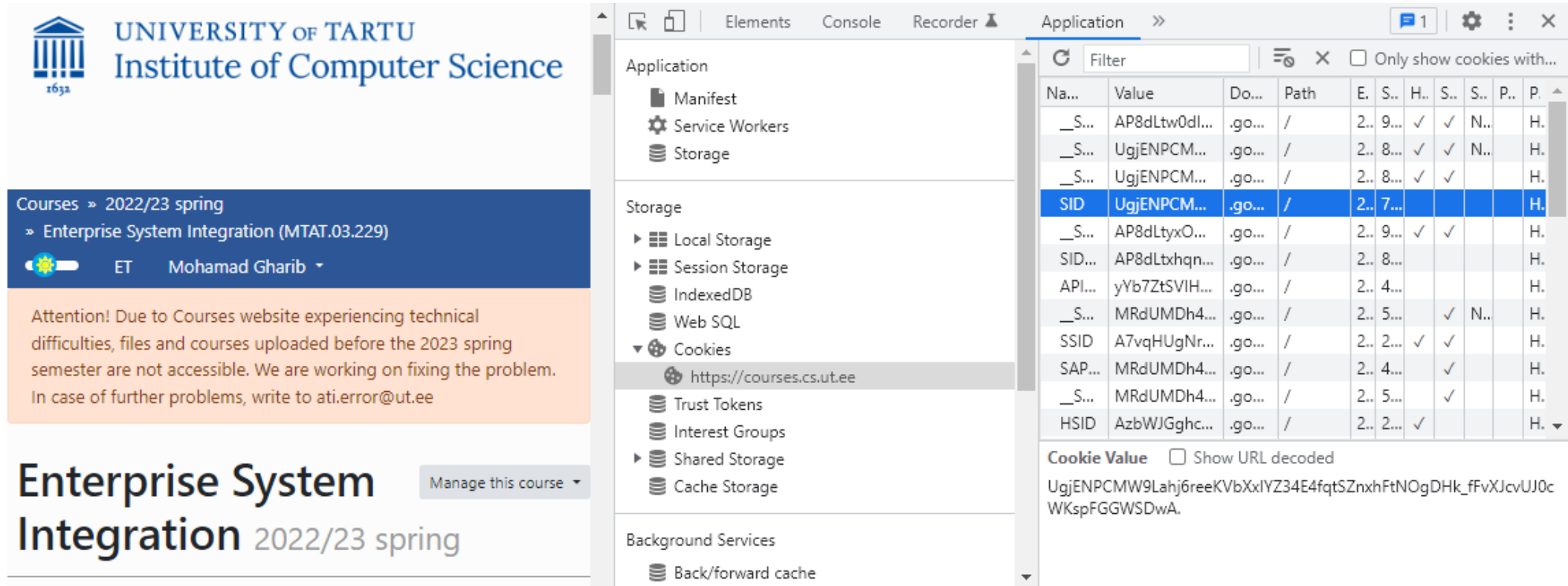
private Boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());}

public Boolean validateToken(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    log.info("extractRoles {} ", extractRoles(token));
    return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));}
```


Tokens/Cookies

Try it yourself ...

Where are these tokens/cookies!



The screenshot shows the University of Tartu website interface on the left and the Chrome DevTools Application tab on the right. The website displays the 'Enterprise System Integration' course for the 2022/23 spring semester. The DevTools Application tab is open to the 'Cookies' section, showing a list of cookies for the domain `https://courses.cs.ut.ee`. The 'SID' cookie is highlighted, and its value is shown in the 'Cookie Value' field.

Na...	Value	Do...	Path	E...	S...	H...	S...	S...	P...	P...
_S...	AP8dLtw0dl...	.go...	/	2..	9...	✓	✓	N..		H.
_S...	UgjENPCM...	.go...	/	2..	8...	✓	✓	N..		H.
_S...	UgjENPCM...	.go...	/	2..	8...	✓	✓			H.
SID	UgjENPCM...	.go...	/	2..	7...					H.
_S...	AP8dLtyxO...	.go...	/	2..	9...	✓	✓			H.
SID...	AP8dLtxhqn...	.go...	/	2..	8...					H.
API...	yYb7ZtSVIH...	.go...	/	2..	4...					H.
_S...	MRdUMDh4...	.go...	/	2..	5...		✓	N..		H.
SSID	A7vqHUgNr...	.go...	/	2..	2...	✓	✓			H.
SAP...	MRdUMDh4...	.go...	/	2..	4...		✓			H.
_S...	MRdUMDh4...	.go...	/	2..	5...		✓			H.
HSID	AzbWJGghc...	.go...	/	2..	2...	✓				H.

Cookie Value ☐ Show URL decoded
 UgjENPCMW9Lahj6reeKVbXxlYZ34E4fqtSZnxhFtNOgDHk_fFvXJcvUJ0cWKspFGGWSDwA.

In the devTool, navigate to the Application tab, then choose “Cookies”, under which you should find the JWT.

Go to <https://courses.cs.ut.ee/2025/esi/spring>, and check how many cookies are placed in your browser 😊

jwt.io

Try it yourself ...



jwt.io*

JWT.IO allows you to decode, verify and generate JWT.

 [Debugger](#) [Libraries](#) [Introduction](#) [Ask](#) Crafted by  auth0 by Okta

Warning: JWTs are credentials, which can grant access to resources. Be careful where you paste them! We do not record tokens, all validation and debugging is done on the client side.

Algorithm HS256

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzkwMjYyLCJ1aWkiOiJkaXZlbnR5cCJ9.wRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD: DATA

```
{  "sub": "1234567890",  "name": "John Doe",  "iat": 1516239022}
```

VERIFY SIGNATURE

```
HMACSHA256(  base64UrlEncode(header) + "." +  base64UrlEncode(payload),  your-256-bit-secret)
```

☐ secret base64 encoded

✔ Signature Verified

SHARE JWT

Warning: JWTs are credentials, which can grant access to resources. Be careful where you paste them! We do not record tokens, all validation and debugging is done on the client side.

* <https://jwt.io/>

100

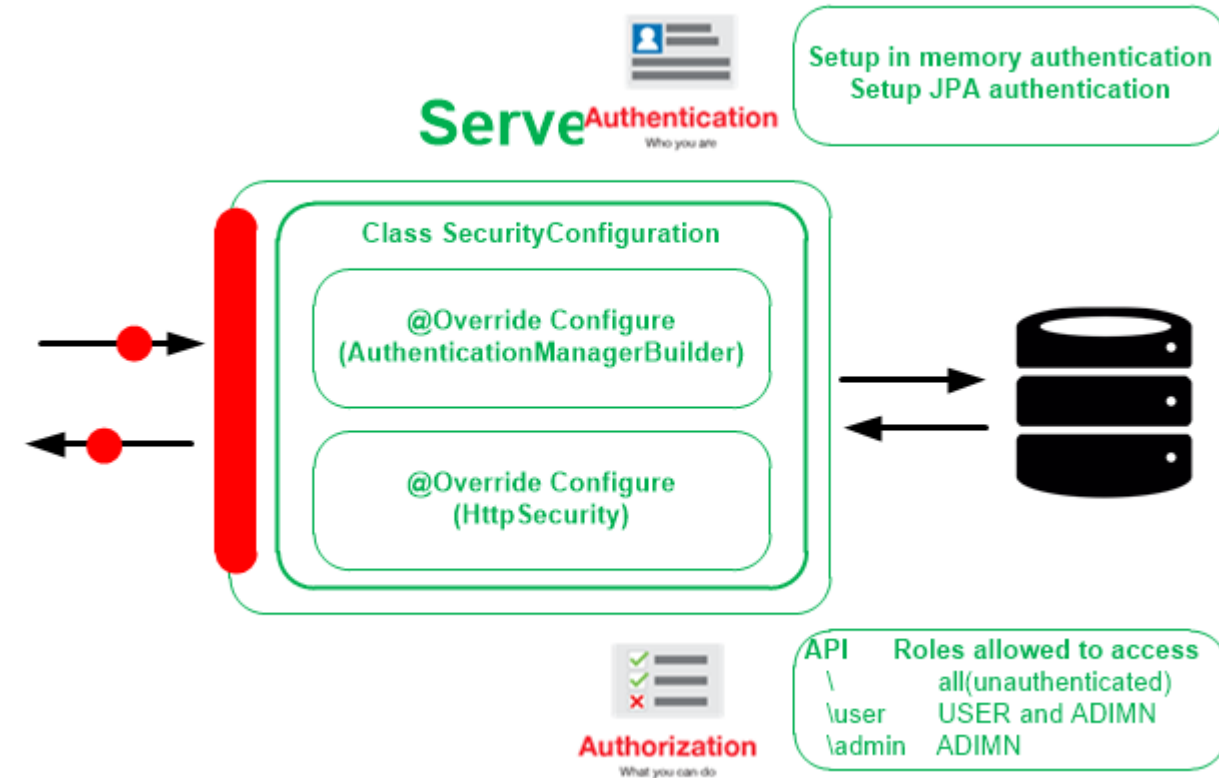
SB Security ...

Try it yourself ...

OAuth2 is an open standard for authorization.

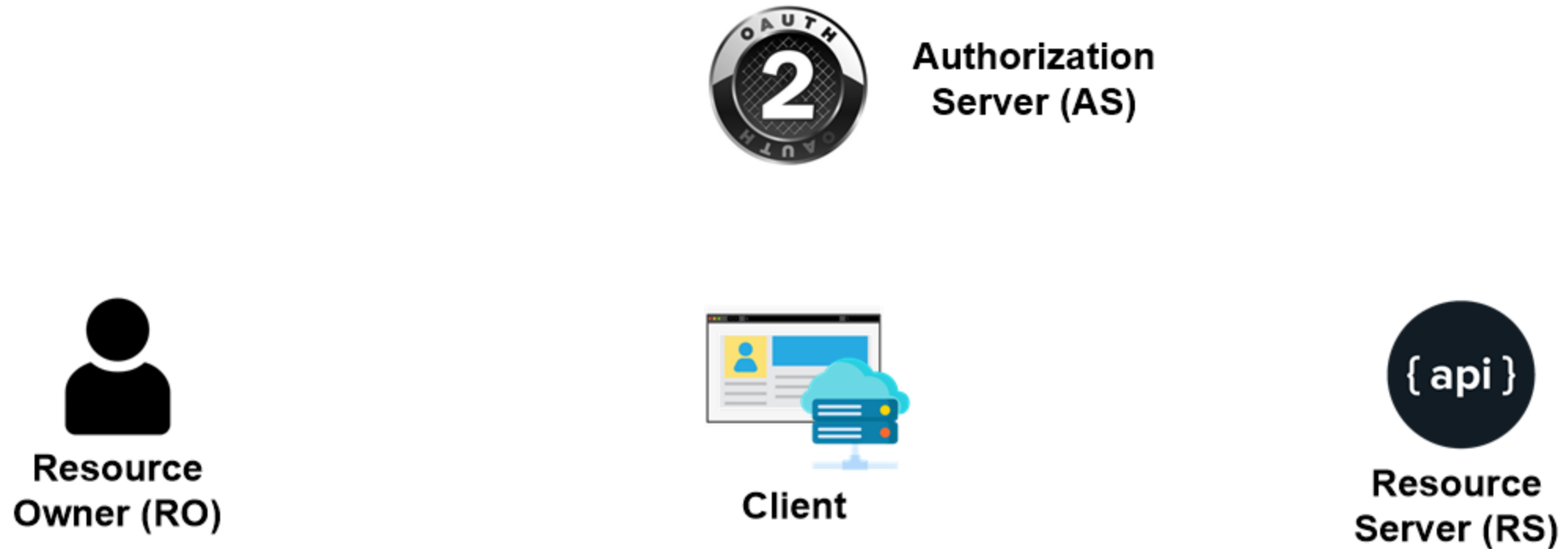
OAuth2 is a standard that apps can use to provide client applications with “secure delegated access”. OAuth2 works over HTTPS and authorizes devices, APIs, servers, and applications with access tokens rather than credentials.

OAuth2 was made famous by HTTP Basic Authentication, where the user is prompted for a username and password.



Does Oauth2 use the same approach?

OAuth2



There are four key actors in *OAuth2* :

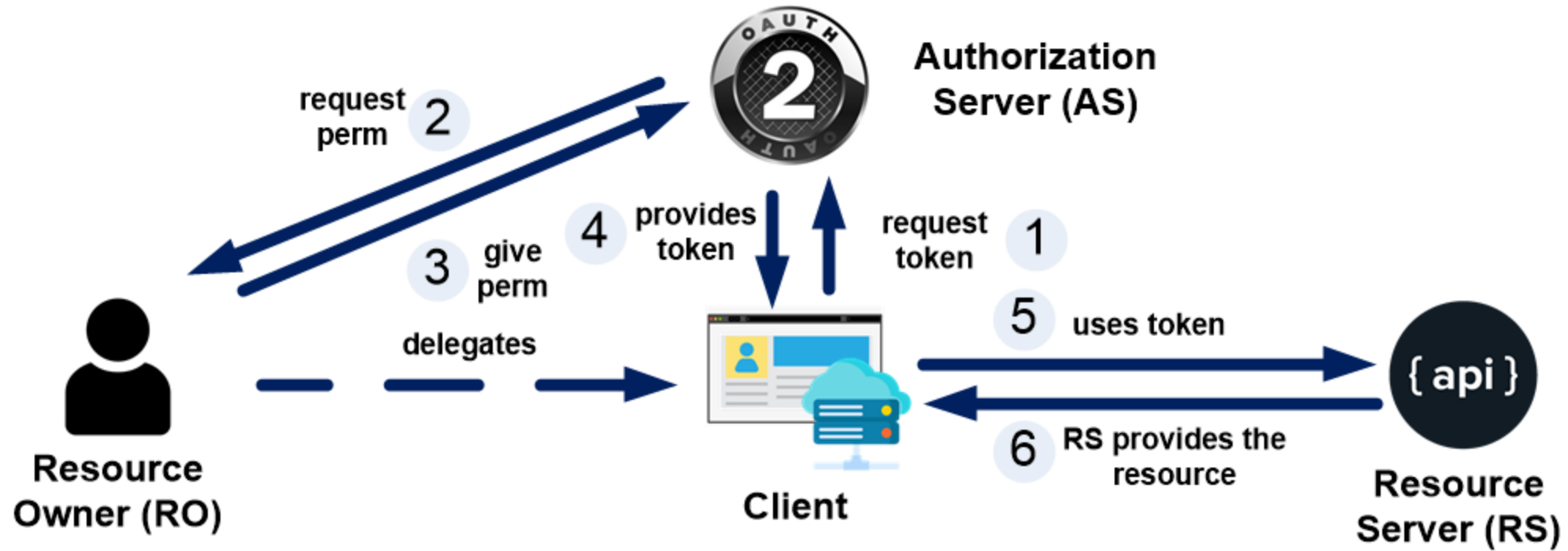
Resource Owner: owns the resource in the resource server.

Resource Server: stores the resource that an application wants to access.

Client: the application that wants to access the resource.

Authorization Server: manages the authentication process.

OAuth2



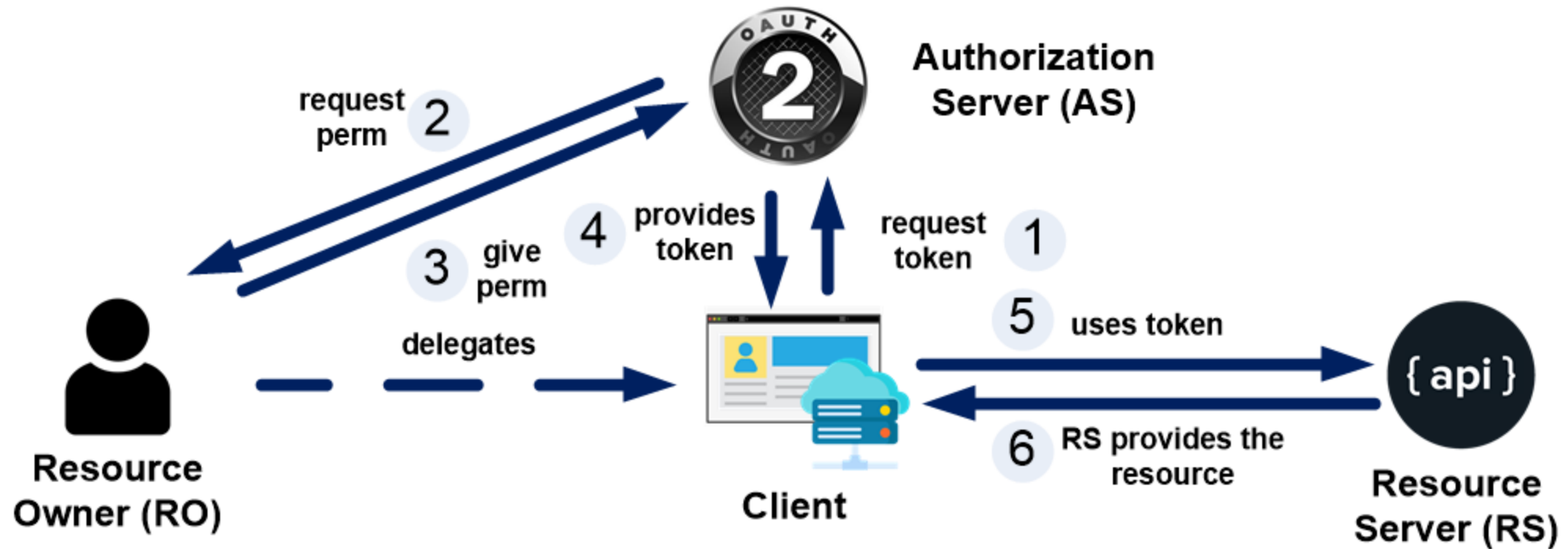
The **Client** sends an **authorization request** to the **Authorization Server**.

The **Authorization Server** requests **permission** from the **owner**.

If the **Owner** consents, the **Authorization Server** provides a token to the **Client**.

The **Client** can use the token to obtain the **Resource** from the **Resource Server**.

OAuth2



In the course Wiki, you can find details on how you can

1. Create a GitHub OAuth2 App, and
2. Configure your App to use it.



UNIVERSITY OF TARTU

**Thank You
for your attention**

Mohamad Gharib
mohamad.gharib@ut.ee



unitartu



tartuuniversity

